

SOLIDWORKS API (.NET VBA C#) — Examples (42/50): Measure_Selected_Entities_Example_VBNET Move_and_Copy_Body_Using_Vertex_Example_

SOLIDWORKS API Documentation (converted from CHM)

Measure Selected Entities Example (VB.NET)

This example shows how to use the measure tool.

```

-----
Preconditions:
1. Verify that the specified part document exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified part document as read only.
2. Selects two faces and measures them.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMeasure As Measure
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\dimxpert\coupling_auto_geo.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_ReadOnly, "", errors, warnings)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("", "FACE", -0.00382117299216134, -0.0032246917626253, -0.00153854043344381, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.00547600669648318, 0.00252191841298099, 0.0050000000000523, True, 0, Nothing, 0)
        swMeasure = swModelDocExt.CreateMeasure
        'Can set this to 0
        ' 0 = center to center
        ' 1 = minimum distance
        ' 2 = maximum distance
        swMeasure.ArcOption = 0

        status = swMeasure.Calculate(Nothing)
        If (status) Then
            If (Not (swMeasure.Length = -1)) Then
                Debug.Print("Length: " & swMeasure.Length)
            End If
            If (Not (swMeasure.Area = -1)) Then
                Debug.Print("Area: " & swMeasure.Area)
            End If
            If (Not (swMeasure.ArcLength = -1)) Then
                Debug.Print("Arc length: " & swMeasure.ArcLength)
            End If
            If (Not (swMeasure.ChordLength = -1)) Then
                Debug.Print("Chord length: " & swMeasure.ChordLength)
            End If
            If (Not (swMeasure.Diameter = -1)) Then
                Debug.Print("Diameter: " & swMeasure.Diameter)
            End If
            If (Not (swMeasure.Radius = -1)) Then
                Debug.Print("Radius: " & swMeasure.Radius)
            End If
            If (Not (swMeasure.Perimeter = -1)) Then
                Debug.Print("Perimeter: " & swMeasure.Perimeter)
            End If
            If (Not (swMeasure.X = -1)) Then
                Debug.Print("X coordinate: " & swMeasure.X)
            End If
            If (Not (swMeasure.Y = -1)) Then
                Debug.Print("Y coordinate: " & swMeasure.Y)
            End If
            If (Not (swMeasure.Z = -1)) Then
                Debug.Print("Z coordinate: " & swMeasure.Z)
            End If
            If (Not (swMeasure.DeltaX = -1)) Then
                Debug.Print("DeltaX: " & swMeasure.DeltaX)
            End If
            If (Not (swMeasure.DeltaY = -1)) Then
                Debug.Print("DeltaY: " & swMeasure.DeltaY)
            End If
            If (Not (swMeasure.DeltaZ = -1)) Then
                Debug.Print("DeltaZ: " & swMeasure.DeltaZ)
            End If
            If (Not (swMeasure.Angle = -1)) Then
                Debug.Print("Angle: " & swMeasure.Angle)
            End If
            If (Not (swMeasure.CenterDistance = -1)) Then
                Debug.Print("Center distance: " & swMeasure.CenterDistance)
            End If
            If (Not (swMeasure.NormalDistance = -1)) Then
                Debug.Print("Normal distance: " & swMeasure.NormalDistance)
            End If
            If (Not (swMeasure.Distance = -1)) Then
                Debug.Print("Distance: " & swMeasure.Distance)
            End If
            If (Not (swMeasure.TotalLength = -1)) Then
                Debug.Print("Total length: " & swMeasure.TotalLength)
            End If
            If (Not (swMeasure.TotalArea = -1)) Then
                Debug.Print("Total area: " & swMeasure.TotalArea)
            End If
            If (swMeasure.IsParallel) Then
                Debug.Print("Is parallel: " & swMeasure.IsParallel)
            End If
            If (swMeasure.IsIntersect) Then
                Debug.Print("Is intersect: " & swMeasure.IsIntersect)
            End If
            If (swMeasure.IsPerpendicular) Then
                Debug.Print("Is perpendicular: " & swMeasure.IsPerpendicular)
            End If
        End If
    End Sub
End Class

```

```

        Debug.Print("Is perpendicular: " & swMeasure.IsPerpendicular)
    End If
    If (Not (swMeasure.Projection = -1)) Then
        Debug.Print("Projection: " & swMeasure.Projection)
    End If
    If (Not (swMeasure.Normal = -1)) Then
        Debug.Print("Normal: " & swMeasure.Normal)
    End If
    If (Not (swMeasure.SphericalCenterDistance = -1)) Then
        Debug.Print("Spherical center distance: " & swMeasure.SphericalCenterDistance)
    End If
    If (swMeasure.IsConcentricSpheres) Then
        Debug.Print("Are concentric spheres: " & swMeasure.IsConcentricSpheres)
    End If
Else
    Debug.Print("Invalid combination of selected entities.")
End If
swModel.ClearSelection2(True)
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

```
End Class
```

Merge and Unmerge Bend Tags Example (C#)

This example shows how to merge and unmerge bend tags in a drawing.

```
//-----
// Preconditions:
// 1. The specified drawing document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. The specified drawing document opens.
// 2. Press F5 repeatedly after examining the changes
//    in the bend tags and the output in the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace MergeBendTagsCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Note swNote = default(Note);
            View swView = default(View);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            Note[] noteList = new Note[2];

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.slddrw";
            swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            System.Diagnostics.Debugger.Break();
            // Locate the bend tags (A, B, C, and D)
            // in the drawing
            // Press F5 to continue

            // Select a bend tag to merge
            status = swModelDocExt.SelectByID2("DetailItem37@Drawing View1", "NOTE", 0.0902750427561398, 0.24484926035503, 0, false, 0, null, 0);
            swNote = (Note)swSelectionMgr.GetSelectedObject6(1, 0);
            noteList[0] = swNote;
            Debug.Print("Is a bendline note? " + swNote.IsBendLineNote);
            swModel.ClearSelection2(true);

            // Select another bend tag with which to merge
            // the previously selected bend tag
            status = swModelDocExt.SelectByID2("DetailItem38@Drawing View1", "NOTE", 0.0978933563656073, 0.244401124260355, 0, true, 0, null, 0);
            swNote = (Note)swSelectionMgr.GetSelectedObject6(1, 0);
            noteList[1] = swNote;
            Debug.Print("Is a bendline note? " + swNote.IsBendLineNote);
            swModel.ClearSelection2(true);

            // Select the drawing view in which to
            // merge bend tags
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.0765893917313017, 0.16302919597189, 0, false, 0, null, 0);
            swView = (View)swSelectionMgr.GetSelectedObject6(1, 0);
            swModel.ClearSelection2(true);

            //Merge the selected bend tags
            status = swView.MergeBendTags(true, noteList);

            swModel.EditRebuild3();

            System.Diagnostics.Debugger.Break();
            // Verify that bend tag A and B merged into
            // bend tag A, bend tag C was renamed to B,
            // and bend tag D was renamed to C
            // Press F5 to continue

            // Select the merged bend tag
            swDrawingDoc = (DrawingDoc)swModel;
            status = swDrawingDoc.ActivateView("Drawing View1");
            status = swModelDocExt.SelectByID2("DetailItem38@Drawing View1", "NOTE", 0.098037379978424, 0.245097849056604, 0, false, 0, null, 0);
            swNote = (Note)swSelectionMgr.GetSelectedObject6(1, 0);
            noteList[0] = swNote;

            // Unmerge the merged bend tag
            swView.MergeBendTags(false, noteList);

            System.Diagnostics.Debugger.Break();
            // Verify that bend tag A and B unmerged,
            // bend tag B was renamed back to C, and bend tag C
            // was renamed back to D
            // Press F5 to finish
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
```


Merge and Unmerge Bend Tags Example (VBA)

This example shows how to merge and unmerge bend tags in a drawing.

```

'-----
' Preconditions:
' 1. The specified drawing document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. The specified drawing document opens.
' 2. Press F5 repeatedly after examining the changes
'    in the bend tags and the output in the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swNote As SldWorks.note
Dim swView As SldWorks.view
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim fileName As String
Dim status As Boolean
Dim errors As Long, warnings As Long
Dim noteList(1) As SldWorks.note

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.slddrw"
    swApp.OpenDoc6 fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings
    Set swModel = swApp.ActiveDoc
    Set swSelectionMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    Stop
    ' Locate the bend tags (A, B, C, and D)
    ' in the drawing
    ' Press F5 to continue

    ' Select a bend tag to merge
    status = swModelDocExt.SelectByID2("DetailItem37@Drawing View1", "NOTE", 9.02750427561398E-02, 0.24484926035503, 0, False, 0,
Nothing, 0)
    Set swNote = swSelectionMgr.GetSelectedObject6(1, 0)
    Set noteList(0) = swNote
    Debug.Print ("Is a bendline note? " & swNote.IsBendLineNote)
    swModel.ClearSelection2 True

    ' Select another bend tag with which to merge
    ' the previously selected bend tag
    status = swModelDocExt.SelectByID2("DetailItem38@Drawing View1", "NOTE", 9.78933563656073E-02, 0.244401124260355, 0, True, 0,
Nothing, 0)
    Set swNote = swSelectionMgr.GetSelectedObject6(1, 0)
    Set noteList(1) = swNote
    Debug.Print ("Is a bendline note? " & swNote.IsBendLineNote)
    swModel.ClearSelection2 True

    ' Select the drawing view in which to
    ' merge bend tags
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 7.65893917313017E-02, 0.16302919597189, 0, False, 0, Nothing,
0)
    Set swView = swSelectionMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True

    'Merge the selected bend tags
    status = swView.MergeBendTags(True, noteList)

    swModel.EditRebuild3

    Stop
    ' Verify that bend tag A and B merged into
    ' bend tag A, bend tag C was renamed to B,
    ' and bend tag D was renamed to C
    ' Press F5 to continue

    ' Select the merged bend tag
    Set swDrawingDoc = swModel
    status = swDrawingDoc.ActivateView("Drawing View1")
    status = swModelDocExt.SelectByID2("DetailItem38@Drawing View1", "NOTE", 0.098037379978424, 0.245097849056604, 0, False, 0,
Nothing, 0)
    Set swNote = swSelectionMgr.GetSelectedObject6(1, 0)
    set noteList(0) = swNote

    ' Unmerge the merged bend tag
    swView.MergeBendTags False, noteList

    Stop

```

```
' Verify that bend tag A and B unmerged,  
' bend tag B was renamed back to C, and bend tag C  
' was renamed back to D  
' Press F5 to finish
```

End Sub

Merge and Unmerge Bend Tags Example (VB.NET)

This example shows how to merge and unmerge bend tags in a drawing.

```

'-----
' Preconditions:
' 1. The specified drawing document exists.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. The specified drawing document opens.
' 2. Press F5 repeatedly after examining the changes
'    in the bend tags and the output in the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swNote As Note
        Dim swView As View
        Dim swDrawingDoc As DrawingDoc
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer, warnings As Integer
        Dim noteList(1) As Note

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.slddrw"
        swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel = swApp.ActiveDoc
        swSelectionMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension

        Stop
        ' Locate the bend tags (A, B, C, and D)
        ' in the drawing
        ' Press F5 to continue

        ' Select a bend tag to merge
        status = swModelDocExt.SelectByID2("DetailItem37@Drawing View1", "NOTE", 0.0902750427561398, 0.24484926035503, 0, False, 0, Nothing, 0)
        swNote = swSelectionMgr.GetSelectedObject6(1, 0)
        noteList(0) = swNote
        Debug.Print("Is a bendline note? " & swNote.IsBendLineNote)
        swModel.ClearSelection2(True)

        ' Select another bend tag with which to merge
        ' the previously selected bend tag
        status = swModelDocExt.SelectByID2("DetailItem38@Drawing View1", "NOTE", 0.0978933563656073, 0.244401124260355, 0, True, 0, Nothing, 0)
        swNote = swSelectionMgr.GetSelectedObject6(1, 0)
        noteList(1) = swNote
        Debug.Print("Is a bendline note? " & swNote.IsBendLineNote)
        swModel.ClearSelection2(True)

        ' Select the drawing view in which to
        ' merge bend tags
        status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.0765893917313017, 0.16302919597189, 0, False, 0, Nothing, 0)
        swView = swSelectionMgr.GetSelectedObject6(1, 0)
        swModel.ClearSelection2(True)

        ' Merge the selected bend tags
        status = swView.MergeBendTags(True, noteList)

        swModel.EditRebuild3()

        Stop
        ' Verify that bend tag A and B merged into
        ' bend tag A, bend tag C was renamed to B,
        ' and bend tag D was renamed to C
        ' Press F5 to continue

        ' Select the merged bend tag
        swDrawingDoc = swModel
        status = swDrawingDoc.ActivateView("Drawing View1")
        status = swModelDocExt.SelectByID2("DetailItem38@Drawing View1", "NOTE", 0.098037379978424, 0.245097849056604, 0, False, 0, Nothing, 0)
        swNote = swSelectionMgr.GetSelectedObject6(1, 0)
        noteList(0) = swNote

        ' Unmerge the merged bend tag
        swView.MergeBendTags(False, noteList)

        Stop
        ' Verify that bend tag A and B unmerged,
        ' bend tag B was renamed back to C, and bend tag C
        ' was renamed back to D
        ' Press F5 to finish

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Merge Arc Segment Bodies With Adjacent Bodies Example (C#)

This example shows how to create structural-member groups with and without merging arc segment bodies with adjacent bodies.

```
//-----
// Preconditions:
// 1. Verify that the specified files exist:
//    * part template
//    * weldment profile
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part.
// 2. Creates a sketch of two lines and two tangent arcs.
// 3. Creates a structural-member group using an adjacent line and arc
//    and merges the arc segment's body with the line's body.
// 4. Creates another structural-member group using the other adjacent
//    line and arc and does not merge the arc segment's body with the
//    line's body.
// 5. Examine the Immediate window.
// 6. Expand Cut list(3) in the FeatureManager design tree.
// 7. Point at each PIPE, SCH 40, 12.70 DIA. and examine the
//    graphics area to verify whether that PIPE, SCH 40, 12.70
//    DIA. is merged.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureManager = default(FeatureManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            StructuralMemberGroup group1 = default(StructuralMemberGroup);
            StructuralMemberGroup group2 = default(StructuralMemberGroup);
            StructuralMemberGroup group = default(StructuralMemberGroup);
            StructuralMemberFeatureData swStructuralMemberFeatureData = default(StructuralMemberFeatureData);
            SketchSegment[] segmentsArray = new SketchSegment[2];
            bool status = false;
            DispatcherWrapper[] groups = new DispatcherWrapper[1];
            object[] groupArray = new object[1];
            int i = 0;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            //Insert weldment feature
            swFeature = (Feature)swFeatureManager.InsertWeldmentFeature();

            //Create sketch of two lines and two tangent arcs
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.0, 0.0, 0.0, -0.101812, 0.0, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.1016, -0.059455, 0.0, 0.0, -0.059455, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateTangentArc(0.0, -0.059455, 0.0, 0.0, 0.0, 1);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateTangentArc(-0.1016, -0.0, 0.0, -0.1016, -0.059455, 0.0, 1);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            swModel.ViewZoomToFit2();
            swModel.ShowNamedView2("Normal To", -1);
            swModel.ClearSelection2(true);

            //Create structural-member group
            group1 = (StructuralMemberGroup)swFeatureManager.CreateStructuralMemberGroup();
            status = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.0963105065508915, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Arc1@Sketch1", "EXTSKETCHSEGMENT", -0.0072699684110568, -0.000902652809559659, 0, true, 0, null, 0);
            segmentsArray[0] = (SketchSegment)swSelectionMgr.GetSelectedObject6(1, -1);
            segmentsArray[1] = (SketchSegment)swSelectionMgr.GetSelectedObject6(2, -1);
            group1.Segments = segmentsArray;
            group1.ApplyCornerTreatment = true;
            group1.CornerTreatmentType = 1;
            group1.GapWithinGroup = 0;
            group1.GapForOtherGroups = 0;
            group1.Angle = 0;
            group1.MergeArcSegmentBodies = true;
            groupArray[0] = group1;
            groups[0] = new DispatcherWrapper(groupArray[0]);
            swFeature = (Feature)swFeatureManager.InsertStructuralWeldment5("C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\lang\\english\\weldment profiles\\ansi inch\\pipe\\0.5 sch 40.sldlfp", (int)swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, false, (groups), "");
            swModel.ClearSelection2(true);

            //Create structural-member group
            group2 = (StructuralMemberGroup)swFeatureManager.CreateStructuralMemberGroup();
            status = swModelDocExt.SelectByID2("Arc2@Sketch1", "EXTSKETCHSEGMENT", -0.106961319560779, -0.000449372254001996, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.0425304114129424, -0.059455, 0, true, 0, null, 0);
            segmentsArray[0] = (SketchSegment)swSelectionMgr.GetSelectedObject6(1, -1);
            segmentsArray[1] = (SketchSegment)swSelectionMgr.GetSelectedObject6(2, -1);
            group2.Segments = segmentsArray;
            group2.ApplyCornerTreatment = true;
            group2.CornerTreatmentType = 1;
            group2.GapWithinGroup = 0;
            group2.GapForOtherGroups = 0;
        }
    }
}
```

```

group2.Angle = 0;
group2.MergeArcSegmentBodies = false;
groupArray[0] = group2;
groups[0] = new DispatcherWrapper(groupArray[0]);
swFeature = (Feature)swFeatureManager.InsertStructuralWeldment5("C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\lang\\english\\weldment profile\\ansi inch\\pipe\\0.5 sch 40.sldlfp", (int)swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, false, (groups), "");
swModel.ClearSelection2(true);

status = swModelDocExt.SelectByID("pipe 0.5 sch 40(1)", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
swStructuralMemberFeatureData = (StructuralMemberFeatureData)swFeature.GetDefinition();
swStructuralMemberFeatureData.AccessSelections(swModel, null);
Debug.Print("");
Debug.Print("Number of groups: " + swStructuralMemberFeatureData.GetGroupsCount());
Debug.Print("Feature name: " + swFeature.Name);
groupArray = (object[])swStructuralMemberFeatureData.Groups;
for (i = 0; i < groupArray.Length; i++)
{
    group = (StructuralMemberGroup)groupArray[i];
    Debug.Print("Arc segment merged? " + group.MergeArcSegmentBodies);
}
swStructuralMemberFeatureData.ReleaseSelectionAccess();

Debug.Print("");

status = swModelDocExt.SelectByID("pipe 0.5 sch 40(2)", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
swStructuralMemberFeatureData = (StructuralMemberFeatureData)swFeature.GetDefinition();
swStructuralMemberFeatureData.AccessSelections(swModel, null);
Debug.Print("");
Debug.Print("Number of groups: " + swStructuralMemberFeatureData.GetGroupsCount());
Debug.Print("Feature name: " + swFeature.Name);
groupArray = (object[])swStructuralMemberFeatureData.Groups;
for (i = 0; i < groupArray.Length; i++)
{
    group = (StructuralMemberGroup)groupArray[i];
    Debug.Print("Arc segment merged? " + group.MergeArcSegmentBodies);
}
swStructuralMemberFeatureData.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Merge Arc Segment Bodies With Adjacent Bodies Example (VBA)

This example shows how to create structural-member groups with and without merging arc segment bodies with adjacent bodies.

```

'-----
' Preconditions:
' 1. Verify that the specified files exist:
'    * part template
'    * weldment profile
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates a sketch of two lines and two tangent arcs.
' 3. Creates a structural-member group using an adjacent line and arc
'    and merges the arc segment's body with the line's body.
' 4. Creates another structural-member group using the other adjacent
'    line and arc and does not merge the arc segment's body with the
'    line's body.
' 5. Examine the Immediate window.
' 6. Expand Cut list(3) in the FeatureManager design tree.
' 7. Point at each PIPE, SCH 40, 12.70 DIA. and examine the
'    graphics area to verify whether that PIPE, SCH 40, 12.70
'    DIA. is merged.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeature As SldWorks.Feature
Dim swFeatureManager As SldWorks.FeatureManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim group1 As SldWorks.StructuralMemberGroup
Dim group2 As SldWorks.StructuralMemberGroup
Dim group As SldWorks.StructuralMemberGroup
Dim swStructuralMemberFeatureData As SldWorks.StructuralMemberFeatureData
Dim segmentsArray(1) As SldWorks.SketchSegment
Dim status As Boolean
Dim groups As Variant
Dim groupArray(0) As Object
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swModelDocExt = swModel.Extension
    Set swSketchMgr = swModel.SketchManager
    Set swSelectionMgr = swModel.SelectionManager

    'Insert weldment feature
    Set swFeature = swFeatureManager.InsertWeldmentFeature

    'Create sketch of two lines and two tangent arcs
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateLine(0#, 0#, 0#, -0.101812, 0#, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.1016, -0.059455, 0#, 0#, -0.059455, 0#)
    Set swSketchSegment = swSketchMgr.CreateTangentArc(0#, -0.059455, 0#, 0#, 0#, 0#, 1)
    Set swSketchSegment = swSketchMgr.CreateTangentArc(-0.1016, -0#, 0#, -0.1016, -0.059455, 0#, 1)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True
    swModel.ViewZoomtofit2
    swModel.ShowNamedView2 "Normal To", -1
    swModel.ClearSelection2 True

    'Create structural-member group
    Set group1 = swFeatureManager.CreateStructuralMemberGroup()
    status = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -9.63105065508915E-02, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Arc1@Sketch1", "EXTSKETCHSEGMENT", 7.2699684110568E-03, -9.02652809559659E-04, 0, True, 0, Nothing, 0)
    Set segmentsArray(0) = swSelectionMgr.GetSelectedObject6(1, -1)
    Set segmentsArray(1) = swSelectionMgr.GetSelectedObject6(2, -1)
    group1.segments = segmentsArray
    group1.ApplyCornerTreatment = True
    group1.CornerTreatmentType = 1
    group1.GapWithinGroup = 0
    group1.GapForOtherGroups = 0
    group1.Angle = 0
    group1.MergeArcSegmentBodies = True
    Set groupArray(0) = group1
    groups = groupArray
    Set swFeature = swFeatureManager.InsertStructuralWeldments5("C:\Program Files\SolidWorks Corp\SOLIDWORKS\lang\english\weldment profiles\ansi
inch\pipe\0.5 sch 40.sldlfp", swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, False, (groups), "")
    swModel.ClearSelection2 True

    'Create structural-member group
    Set group2 = swFeatureManager.CreateStructuralMemberGroup()
    status = swModelDocExt.SelectByID2("Arc2@Sketch1", "EXTSKETCHSEGMENT", -0.106961319560779, -4.49372254001996E-04, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -4.25304114129424E-02, -0.059455, 0, True, 0, Nothing, 0)
    Set segmentsArray(0) = swSelectionMgr.GetSelectedObject6(1, -1)
    Set segmentsArray(1) = swSelectionMgr.GetSelectedObject6(2, -1)

```

```

group2.segments = segmentsArray
group2.ApplyCornerTreatment = True
group2.CornerTreatmentType = 1
group2.GapWithinGroup = 0
group2.GapForOtherGroups = 0
group2.Angle = 0
group2.MergeArcSegmentBodies = False
Set groupArray(0) = group2
groups = groupArray
Set swFeature = swFeatureManager.InsertStructuralWeldments5("C:\Program Files\SolidWorks Corp\SOLIDWORKS\lang\english\weldment profiles\ansi
inch\pipe\0.5 sch 40.sldlfp", swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, False, (groups), "")
swModel.ClearSelection2 True

status = swModelDocExt.SelectByID("pipe 0.5 sch 40(1)", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
Set swStructuralMemberFeatureData = swFeature.GetDefinition
swStructuralMemberFeatureData.AccessSelections swModel, Nothing
Debug.Print ""
Debug.Print "Number of groups: " & swStructuralMemberFeatureData.GetGroupsCount
Debug.Print " Feature name: " & swFeature.Name
groups = swStructuralMemberFeatureData.groups
For i = LBound(groups) To UBound(groups)
    Set group = groups(i)
    Debug.Print (" Arc segment merged? " & group.MergeArcSegmentBodies)
Next i
swStructuralMemberFeatureData.ReleaseSelectionAccess

Debug.Print ""

status = swModelDocExt.SelectByID("pipe 0.5 sch 40(2)", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
Set swStructuralMemberFeatureData = swFeature.GetDefinition
swStructuralMemberFeatureData.AccessSelections swModel, Nothing
Debug.Print ""
Debug.Print "Number of groups: " & swStructuralMemberFeatureData.GetGroupsCount
Debug.Print " Feature name: " & swFeature.Name
groups = swStructuralMemberFeatureData.groups
For i = LBound(groups) To UBound(groups)
    Set group = groups(i)
    Debug.Print (" Arc segment merged? " & group.MergeArcSegmentBodies)
Next i
swStructuralMemberFeatureData.ReleaseSelectionAccess

End Sub

```

Merge Arc Segment Bodies With Adjacent Bodies Example (VB.NET)

This example shows how to create structural-member groups with and without merging arc segment bodies with adjacent bodies.

```

-----
Preconditions:
1. Verify that the specified files exist:
    * part template
    * weldment profile
2. Open the Immediate window.
Postconditions:
1. Opens a new part.
2. Creates a sketch of two lines and two tangent arcs.
3. Creates a structural-member group using an adjacent line and arc
   and merges the arc segment's body with the line's body.
4. Creates another structural-member group using the other adjacent
   line and arc and does not merge the arc segment's body with the
   line's body.
5. Examine the Immediate window.
6. Expand Cut List(3) in the FeatureManager design tree.
7. Point at each PIPE, SCH 40, 12.70 DIA, and examine the
   graphics area to verify whether that PIPE, SCH 40, 12.70
   DIA, is merged.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swFeature As Feature
        Dim swFeatureManager As FeatureManager
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swSelectionMgr As SelectionMgr
        Dim group1 As StructuralMemberGroup
        Dim group2 As StructuralMemberGroup
        Dim group As StructuralMemberGroup
        Dim swStructuralMemberFeatureData As StructuralMemberFeatureData
        Dim segmentsArray(1) As SketchSegment
        Dim status As Boolean
        Dim groups(0) As DispatcherWrapper
        Dim groupArray(0) As Object
        Dim i As Integer

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swFeatureManager = swModel.FeatureManager
        swModelDocExt = swModel.Extension
        swSketchMgr = swModel.SketchManager
        swSelectionMgr = swModel.SelectionManager

        'Insert weldment feature
        swFeature = swFeatureManager.InsertWeldmentFeature

        'Create sketch of two lines and two tangent arcs
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        swSketchSegment = swSketchMgr.CreateLine(0.0#, 0.0#, 0.0#, -0.101812, 0.0#, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(-0.1016, -0.059455, 0.0#, 0.0#, -0.059455, 0.0#)
        swSketchSegment = swSketchMgr.CreateTangentArc(0.0#, -0.059455, 0.0#, 0.0#, 0.0#, 0.0#, 1)
        swSketchSegment = swSketchMgr.CreateTangentArc(-0.1016, -0.0#, 0.0#, -0.1016, -0.059455, 0.0#, 1)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        swModel.ViewZoomtofit2()
        swModel.ShowNamedView2("Normal To", -1)
        swModel.ClearSelection2(True)

        'Create structural-member group
        group1 = swFeatureManager.CreateStructuralMemberGroup()
        status = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.0963105065508915, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Arc1@Sketch1", "EXTSKETCHSEGMENT", 0.0072699684110568, -0.000902652809559659, 0, True, 0, Nothing, 0)
        segmentsArray(0) = swSelectionMgr.GetSelectedObject6(1, -1)
        segmentsArray(1) = swSelectionMgr.GetSelectedObject6(2, -1)
        group1.Segments = segmentsArray
        group1.ApplyCornerTreatment = True
        group1.CornerTreatmentType = 1
        group1.GapWithinGroup = 0
        group1.GapForOtherGroups = 0
        group1.Angle = 0
        group1.MergeArcSegmentBodies = True
        groupArray(0) = group1
        groups(0) = New DispatcherWrapper(groupArray(0))
        swFeature = swFeatureManager.InsertStructuralWeldment5("C:\Program Files\SolidWorks Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\pipe\
0.5 sch 40.sldlfp", swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, False, (groups), "")
        swModel.ClearSelection2(True)

        'Create structural-member group
        group2 = swFeatureManager.CreateStructuralMemberGroup()
        status = swModelDocExt.SelectByID2("Arc2@Sketch1", "EXTSKETCHSEGMENT", -0.106961319560779, -0.000449372254001996, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.0425304114129424, -0.059455, 0, True, 0, Nothing, 0)
        segmentsArray(0) = swSelectionMgr.GetSelectedObject6(1, -1)
        segmentsArray(1) = swSelectionMgr.GetSelectedObject6(2, -1)
        group2.Segments = segmentsArray
        group2.ApplyCornerTreatment = True
        group2.CornerTreatmentType = 1
        group2.GapWithinGroup = 0
        group2.GapForOtherGroups = 0
        group2.Angle = 0
        group2.MergeArcSegmentBodies = False
        groupArray(0) = group2
        groups(0) = New DispatcherWrapper(groupArray(0))
        swFeature = swFeatureManager.InsertStructuralWeldment5("C:\Program Files\SolidWorks Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\pipe\
0.5 sch 40.sldlfp", swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, False, (groups), "")
    
```

```

swModel.ClearSelection2(True)

status = swModelDocExt.SelectByID("pipe 0.5 sch 40(1)", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
swStructuralMemberFeatureData = swFeature.GetDefinition
swStructuralMemberFeatureData.AccessSelections(swModel, Nothing)
Debug.Print("")
Debug.Print("Number of groups: " & swStructuralMemberFeatureData.GetGroupsCount)
Debug.Print(" Feature name: " & swFeature.Name)
groupArray = swStructuralMemberFeatureData.Groups
For i = LBound(groupArray) To UBound(groupArray)
    group = groupArray(i)
    Debug.Print(" Arc segment merged? " & group.MergeArcSegmentBodies)
Next i
swStructuralMemberFeatureData.ReleaseSelectionAccess()

Debug.Print("")

status = swModelDocExt.SelectByID("pipe 0.5 sch 40(2)", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
swStructuralMemberFeatureData = swFeature.GetDefinition
swStructuralMemberFeatureData.AccessSelections(swModel, Nothing)
Debug.Print("")
Debug.Print("Number of groups: " & swStructuralMemberFeatureData.GetGroupsCount)
Debug.Print(" Feature name: " & swFeature.Name)
groupArray = swStructuralMemberFeatureData.Groups
For i = LBound(groupArray) To UBound(groupArray)
    group = groupArray(i)
    Debug.Print(" Arc segment merged? " & group.MergeArcSegmentBodies)
Next i
swStructuralMemberFeatureData.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Merge Miter Trimmed Bodies Example (C#)

This example shows how to specify to merge miter trimmed bodies in a structural-member group.

```
//-----
// Preconditions:
// 1. Verify that the specified weldment profile exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a new document containing a weldment and a structural member.
// 2. Sets to merge miter trimmed bodies in the structural-member group.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatMgr = default(FeatureManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swWeldFeat = default(Feature);
            StructuralMemberFeatureData swWeldFeatData = default(StructuralMemberFeatureData);
            Feature swFeature = default(Feature);
            bool status = false;
            string template = null;
            object[] skLines = null;
            StructuralMemberGroup group1 = default(StructuralMemberGroup);
            SketchSegment[] segments1 = new SketchSegment[2];
            DispatcherWrapper[] groups1 = new DispatcherWrapper[1];
            object[] groupArray1 = new object[1];
            StructuralMemberGroup group = default(StructuralMemberGroup);
            object[] groups = null;
            object[] segments = null;
            int i = 0;
            int j = 0;

            template = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
            swModel = (ModelDoc2)swApp.NewDocument(template, 0, 0, 0);
            swFeatMgr = (FeatureManager)swModel.FeatureManager;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModel.ClearSelection2(true);

            // Create weldment and structural member
            skLines = (object[])swModel.SketchManager.CreateCornerRectangle(-0.1872393706766, 0.1133237194389, 0, -0.07003610048208, 0.009188409684237, 0)
;

            swModel.ClearSelection2(true);
            swModel.SketchManager.InsertSketch(true);
            swModel.ViewZoomToFit2();
            swFeature = (Feature)swFeatMgr.InsertWeldmentFeature();
            group1 = (StructuralMemberGroup)swFeatMgr.CreateStructuralMemberGroup();
            status = swModel.Extension.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.1495427140733, 0.1133237194389, 0, false, 0, null, 0);
            status = swModel.Extension.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.1872393706766, 0.08238014634844, 0, true, 0, null, 0);
            segments1[0] = (SketchSegment)swSelMgr.GetSelectedObject6(1, 0);
            segments1[1] = (SketchSegment)swSelMgr.GetSelectedObject6(2, 0);
            group1.Segments = segments1;
            group1.Angle = 0;
            group1.ApplyCornerTreatment = true;
            group1.CornerTreatmentType = (int)swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter;
            group1.MirrorProfile = true;
            group1.MirrorProfileAxis = (int)swMirrorProfileOrAlignmentAxis_e.swMirrorProfileOrAlignmentAxis_Veritical;
            group1.GapWithinGroup = 0;
            groupArray1[0] = (object)group1;
            groups1[0] = new DispatcherWrapper(groupArray1[0]);
            swFeature = (Feature)swFeatMgr.InsertStructuralWeldment5("C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\weldment profiles\\an
si inch\\c channel\\3 x 5.sldlfp", (int)swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, false, groups1, "");
            swModel.ClearSelection2(true);

            // Get and set structural-member group information
            status = swModel.Extension.SelectByID2("c channel 3 x 5(1)", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swWeldFeat = (Feature)swSelMgr.GetSelectedObject6(1, 0);
            swWeldFeatData = (StructuralMemberFeatureData)swWeldFeat.GetDefinition();
            swWeldFeatData.AccessSelections(swModel, null);
            Debug.Print("");
            Debug.Print("Number of groups: " + swWeldFeatData.GetGroupsCount());
            Debug.Print("Feature name: " + swWeldFeat.Name);
            groups = (object[])swWeldFeatData.Groups;
            for (i = 0; i < groups.Length; i++)
            {
                group = (StructuralMemberGroup)groups[i];
                Debug.Print(" Number of segments in group " + i + 1 + ": " + group.GetSegmentsCount());
                Debug.Print(" Apply corner treatment? " + group.ApplyCornerTreatment);
                Debug.Print(" Corner treatment type (1 = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter): " + group.CornerTreatmentType);
                Debug.Print(" Original merge miter trimmed bodies setting: " + group.MiterMergeCondition);
                if (group.CornerTreatmentType == (int)swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter)
                {
                    group.MiterMergeCondition = true;
                }
                Debug.Print(" Modified merge miter trimmed bodies setting: " + group.MiterMergeCondition);
                segments = (object[])group.Segments;
                for (j = 0; j < segments.Length; j++)
                {
                    ((SketchSegment)segments[j]).Select4(false, null);
                }
            }
            swFeature.ModifyDefinition(swWeldFeatData, swModel, null);
            swModel.ClearSelection2(true);
        }
    }
}
```

```
    /// <summary>  
    /// The Sldworks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Merge Miter Trimmed Bodies Example (VBA)

This example shows how to specify to merge miter trimmed bodies in a structural-member group.

```

'-----
' Preconditions:
' 1. Verify that the specified weldment profile exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new document containing a weldment and a structural member.
' 2. Sets to merge miter trimmed bodies in the structural-member group.
' 3. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatMgr As SldWorks.FeatureManager
Dim swSelMgr As SldWorks.SelectionMgr
Dim swWeldFeat As SldWorks.Feature
Dim swWeldFeatData As SldWorks.StructuralMemberFeatureData
Dim swFeature As SldWorks.Feature
Dim status As Boolean
Dim template As String
Dim skLines As Variant
Dim group1 As SldWorks.StructuralMemberGroup
Dim seg1 As Variant
Dim segments1(1) As Object
Dim groups1 As Variant
Dim groupArray1(0) As Object
Dim group As SldWorks.StructuralMemberGroup
Dim groups As Variant
Dim segments As Variant
Dim i As Long
Dim j As Long

Sub main()

    Set swApp = Application.SldWorks

    template = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
    Set swModel = swApp.NewDocument(template, 0, 0, 0)
    Set swFeatMgr = swModel.FeatureManager
    Set swSelMgr = swModel.SelectionManager

    swModel.ClearSelection2 True

    ' Create weldment and structural member
    skLines = swModel.SketchManager.CreateCornerRectangle(-0.1872393706766, 0.1133237194389, 0, -0.07003610048208, 0.009188409684237, 0)
    swModel.ClearSelection2 True
    swModel.SketchManager.InsertSketch True
    swModel.ViewZoomToFit2
    Set swFeature = swFeatMgr.InsertWeldmentFeature()
    Set group1 = swFeatMgr.CreateStructuralMemberGroup
    status = swModel.Extension.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.1495427140733, 0.1133237194389, 0, False, 0, Nothing, 0)
    status = swModel.Extension.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.1872393706766, 0.08238014634844, 0, True, 0, Nothing, 0)
    Set segments1(0) = swSelMgr.GetSelectedObject6(1, 0)
    Set segments1(1) = swSelMgr.GetSelectedObject6(2, 0)
    seg1 = segments1
    group1.segments = (seg1)
    group1.Angle = 0
    group1.ApplyCornerTreatment = True
    group1.CornerTreatmentType = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter
    group1.MirrorProfile = True
    group1.MirrorProfileAxis = swMirrorProfileOrAlignmentAxis_e.swMirrorProfileOrAlignmentAxis_Vertical
    group1.GapWithinGroup = 0
    Set groupArray1(0) = group1
    groups1 = groupArray1
    Set swFeature = swFeatMgr.InsertStructuralWeldment5("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\c channel\3
x 5.sldlfp", swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, False, (groups1), "")
    swModel.ClearSelection2 True

    ' Get and set structural-member group information
    status = swModel.Extension.SelectByID2("c channel 3 x 5(1)", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swWeldFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swWeldFeatData = swWeldFeat.GetDefinition
    swWeldFeatData.AccessSelections swModel, Nothing
    Debug.Print ""
    Debug.Print "Number of groups: " & swWeldFeatData.GetGroupsCount
    Debug.Print " Feature name: " & swWeldFeat.Name
    groups = swWeldFeatData.groups
    For i = LBound(groups) To UBound(groups)
        Set group = groups(i)
        Debug.Print " Number of segments in group " & i + 1 & ": " & group.GetSegmentsCount
        Debug.Print " Apply corner treatment? " & group.ApplyCornerTreatment
        Debug.Print " Corner treatment type (1 = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter): " & group.CornerTreatmentType
        Debug.Print " Original merge miter trimmed bodies setting: " & group.MiterMergeCondition
        If group.CornerTreatmentType = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter Then
            group.MiterMergeCondition = True
        End If
        Debug.Print " Modified merge miter trimmed bodies setting: " & group.MiterMergeCondition
        segments = group.segments
        For j = LBound(segments) To UBound(segments)
            segments(j).Select4 False, Nothing
        Next j
    Next i

```

```
        Next i
    swFeature.ModifyDefinition swWeldFeatData, swModel, Nothing
    swModel.ClearSelection2 True
End Sub
```

Merge Miter Trimmed Bodies Example (VB.NET)

This example shows how to specify to merge miter trimmed bodies in a structural-member group.

```

-----
Preconditions:
1. Verify that the specified weldment profile exists.
2. Open the Immediate window.
Postconditions:
1. Creates a new document containing a weldment and a structural member.
2. Sets to merge miter trimmed bodies in the structural-member group.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatMgr As FeatureManager
        Dim swSelMgr As SelectionMgr
        Dim swWeldFeat As Feature
        Dim swWeldFeatData As StructuralMemberFeatureData
        Dim swFeature As Feature
        Dim swSketchSegment As SketchSegment
        Dim status As Boolean
        Dim template As String
        Dim skLines As Object
        Dim group1 As StructuralMemberGroup
        Dim segments1(1) As SketchSegment
        Dim groups1(0) As DispatcherWrapper
        Dim groupArray1(0) As Object
        Dim group As StructuralMemberGroup
        Dim groups As Object
        Dim segments As Object
        Dim i As Integer
        Dim j As Integer

        template = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
        swModel = swApp.NewDocument(template, 0, 0, 0)
        swFeatMgr = swModel.FeatureManager
        swSelMgr = swModel.SelectionManager
        swModel.ClearSelection2(True)

        ' Create weldment and structural member
        skLines = swModel.SketchManager.CreateCornerRectangle(-0.1872393706766, 0.1133237194389, 0, -0.07003610048208, 0.009188409684237, 0)
        swModel.ClearSelection2(True)
        swModel.SketchManager.InsertSketch(True)
        swModel.ViewZoomtofit2()
        swFeature = swFeatMgr.InsertWeldmentFeature()
        group1 = swFeatMgr.CreateStructuralMemberGroup
        status = swModel.Extension.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.1495427140733, 0.1133237194389, 0, False, 0, Nothing, 0)
        status = swModel.Extension.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.1872393706766, 0.08238014634844, 0, True, 0, Nothing, 0)
        segments1(0) = swSelMgr.GetSelectedObject6(1, 0)
        segments1(1) = swSelMgr.GetSelectedObject6(2, 0)
        group1.Segments = segments1
        group1.Angle = 0
        group1.ApplyCornerTreatment = True
        group1.CornerTreatmentType = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter
        group1.MirrorProfile = True
        group1.MirrorProfileAxis = swMirrorProfileOrAlignmentAxis_e.swMirrorProfileOrAlignmentAxis_Veritical
        group1.GapWithinGroup = 0
        groupArray1(0) = group1
        groups1(0) = New DispatcherWrapper(groupArray1(0))
        swFeature = swFeatMgr.InsertStructuralWeldments5("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\c channel\3
x 5.sldlfp", swConnectedSegmentsOption_e.swConnectedSegments_SimpleCut, False, groups1, "")
        swModel.ClearSelection2(True)

        ' Get and set structural-member group information
        status = swModel.Extension.SelectByID2("c channel 3 x 5(1)", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swWeldFeat = swSelMgr.GetSelectedObject6(1, 0)
        swWeldFeatData = swWeldFeat.GetDefinition
        swWeldFeatData.AccessSelections(swModel, Nothing)
        Debug.Print("")
        Debug.Print("Number of groups: " & swWeldFeatData.GetGroupsCount)
        Debug.Print(" Feature name: " & swWeldFeat.Name)
        groups = swWeldFeatData.Groups
        For i = LBound(groups) To UBound(groups)
            group = groups(i)
            Debug.Print(" Number of segments in group " & i + 1 & ": " & group.GetSegmentsCount)
            Debug.Print(" Apply corner treatment? " & group.ApplyCornerTreatment)
            Debug.Print(" Corner treatment type (1 = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter): " & group.CornerTreatmentType)
            Debug.Print(" Original merge miter trimmed bodies setting: " & group.MiterMergeCondition)
            If group.CornerTreatmentType = swSolidworksWeldmentEndCondOptions_e.swEndConditionMiter Then
                group.MiterMergeCondition = True
            End If
            Debug.Print(" Modified merge miter trimmed bodies setting: " & group.MiterMergeCondition)
            segments = group.Segments
            For j = LBound(segments) To UBound(segments)
                swSketchSegment = segments(j)
                swSketchSegment.Select4(False, Nothing)
            Next j
        Next i
        swFeature.ModifyDefinition(swWeldFeatData, swModel, Nothing)

        swModel.ClearSelection2(True)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Mirror Components Example (C#)

This example shows how to mirror components in an assembly.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\advdrawings\98food processor.sldasm.
//
// Postconditions:
// 1. Inserts reference plane PLANE4.
// 2. Creates feature MirrorComponent1 that mirrors six assembly
//    components.
// 3. Saves the mirror components to files with file name suffix _TestMirror to
//    public_documents\samples\tutorial\advdrawings.
// 4. Examine public_documents\samples\tutorial\advdrawings, the FeatureManager design
//    tree, and the graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace MirrorComponents2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            bool boolstatus = false;
            boolstatus = swModel.Extension.SelectByID("", "FACE", 0.104250921669188, -0.000236987012272039, -0.05971999999999418,
true, 0, null, 0);
            RefPlane myRefPlane = default(RefPlane);
            myRefPlane = (RefPlane)swModel.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0);

            AssemblyDoc swAssem = default(AssemblyDoc);
            swAssem = (AssemblyDoc)swModel;

            object compsToInstance = null;
            compsToInstance = null;

            object filenames = null;
            filenames = null;

            string location = null;
            location = "";

            swMirrorComponentNameModifier_e nameModifierType = default(swMirrorComponentNameModifier_e);
            nameModifierType = swMirrorComponentNameModifier_e.swMirrorComponentName_Suffix;
            string nameModifier = null;
            nameModifier = "_TestMirror";

            Feature mirrorPlane = default(Feature);
            mirrorPlane = (Feature)swAssem.FeatureByName("PLANE4");

            Component2[] compsToMirror = new Component2[6];
            compsToMirror[0] = swAssem.GetComponentByName("gear- caddy-1");
            compsToMirror[1] = swAssem.GetComponentByName("middle-gear-1");
            compsToMirror[2] = swAssem.GetComponentByName("shaft gear-1");
            compsToMirror[3] = swAssem.GetComponentByName("middle-gear plate-1");
            compsToMirror[4] = swAssem.GetComponentByName("base plate-1");
            compsToMirror[5] = swAssem.GetComponentByName("shaft gear insert-1");

            object orientations = null;
            orientations = null;

            bool orientAboutCoM = false;
            orientAboutCoM = true;

            bool createDerivedConfigs = false;
            createDerivedConfigs = false;

            int importOptions = 0;
            importOptions = (int)swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids;

            bool breakLinks = false;
            breakLinks = false;
            bool preserveZAxis = false;
            preserveZAxis = true;
        }
    }
}
```

```
        object vResult = null;
        vResult = swAssem.MirrorComponents3(mirrorPlane, compsToInstance, orientations, orientAboutCoM, (compsToMirror),
createDerivedConfigs, filenames, (int)nameModifierType, nameModifier, location, importOptions, breakLinks, preserveZAxis, true);
    }
    public SldWorks swApp;
} }
```

Mirror Components Example (VBA)

This example shows how to mirror components in an assembly.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\98food processor.sldasm.
'
' Postconditions:
' 1. Inserts reference plane PLANE4.
' 2. Creates feature MirrorComponent1 that mirrors six assembly
'    components.
' 3. Saves the mirror components to files with file name suffix _TestMirror to
'    public_documents\samples\tutorial\advdrawings.
' 4. Examine public_documents\samples\tutorial\advdrawings, the FeatureManager design
'    tree, and the graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Sub main()
    Dim swApp As SldWorks.SldWorks
    Set swApp = Application.SldWorks

    Dim swModel As SldWorks.ModelDoc2
    Set swModel = swApp.ActiveDoc

    Dim boolstatus As Boolean
    boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.104250921669188, -2.36987012272039E-04, -5.97199999999418E-02, True,
0, Nothing, 0)
    Dim myRefPlane As SldWorks.RefPlane
    Set myRefPlane = swModel.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0)

    Dim swAssem As SldWorks.AssemblyDoc
    Set swAssem = swModel

    Dim compsToInstance As Object
    Set compsToInstance = Nothing

    Dim filenames As Object
    Set filenames = Nothing

    Dim location As String
    location = ""

    Dim nameModifierType As swMirrorComponentNameModifier_e
    nameModifierType = swMirrorComponentNameModifier_e_Suffix
    Dim nameModifier As String
    nameModifier = "_TestMirror"

    Dim mirrorPlane As SldWorks.Feature
    Set mirrorPlane = swAssem.FeatureByName("PLANE4")

    Dim compsToMirror(0 To 5) As Object
    Set compsToMirror(0) = swAssem.GetComponentByName("gear- caddy-1")
    Set compsToMirror(1) = swAssem.GetComponentByName("middle-gear-1")
    Set compsToMirror(2) = swAssem.GetComponentByName("shaft gear-1")
    Set compsToMirror(3) = swAssem.GetComponentByName("middle-gear plate-1")
    Set compsToMirror(4) = swAssem.GetComponentByName("base plate-1")
    Set compsToMirror(5) = swAssem.GetComponentByName("shaft gear insert-1")

    Dim orientations As Object
    Set orientations = Nothing

    Dim orientAboutCoM As Boolean
    orientAboutCoM = True

    Dim createDerivedConfigs As Boolean
    createDerivedConfigs = False

    Dim importOptions As Long
    importOptions = swMirrorPartOptions_ImportSolids

    Dim breakLinks As Boolean
    breakLinks = False
    Dim preserveZAxis As Boolean
    preserveZAxis = True

    Dim vResult As Variant
    vResult = swAssem.MirrorComponents3(mirrorPlane, compsToInstance, orientations, orientAboutCoM, (compsToMirror),
createDerivedConfigs, filenames, nameModifierType, nameModifier, location, importOptions, breakLinks, preserveZAxis, True)

End Sub

```

Mirror Components Example (VB.NET)

This example shows how to mirror components in an assembly.

```

' Preconditions: Open public_documents\samples\tutorial\advdrawings\98food processor.sldasm.
'
' Postconditions:
' 1. Inserts reference plane PLANE4.
' 2. Creates feature MirrorComponent1 that mirrors six assembly
'    components.
' 3. Saves the mirror components to files with file name suffix _TestMirror to
'    public_documents\samples\tutorial\advdrawings.
' 4. Examine public_documents\samples\tutorial\advdrawings, the FeatureManager design
'    tree, and the graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```

Partial Class SolidWorksMacro

```

```

    Sub main()

        Dim swModel As ModelDoc2
        swModel = swApp.ActiveDoc

        Dim boolstatus As Boolean
        boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.104250921669188, -0.000236987012272039, -0.0597199999999418, True,
0, Nothing, 0)
        Dim myRefPlane As RefPlane
        myRefPlane = swModel.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0)

        Dim swAssem As AssemblyDoc
        swAssem = swModel

        Dim compsToInstance As Object
        compsToInstance = Nothing

        Dim filenames As Object
        filenames = Nothing

        Dim location As String
        location = ""

        Dim nameModifierType As swMirrorComponentNameModifier_e
        nameModifierType = swMirrorComponentNameModifier_e.swMirrorComponentName_Suffix
        Dim nameModifier As String
        nameModifier = "_TestMirror"

        Dim mirrorPlane As Feature
        mirrorPlane = swAssem.FeatureByName("PLANE4")

        Dim compsToMirror(0 To 5) As Component2
        compsToMirror(0) = swAssem.GetComponentByName("gear- caddy-1")
        compsToMirror(1) = swAssem.GetComponentByName("middle-gear-1")
        compsToMirror(2) = swAssem.GetComponentByName("shaft gear-1")
        compsToMirror(3) = swAssem.GetComponentByName("middle-gear plate-1")
        compsToMirror(4) = swAssem.GetComponentByName("base plate-1")
        compsToMirror(5) = swAssem.GetComponentByName("shaft gear insert-1")

        Dim orientations As Object
        orientations = Nothing

        Dim orientAboutCoM As Boolean
        orientAboutCoM = True

        Dim createDerivedConfigs As Boolean
        createDerivedConfigs = False

        Dim importOptions As Integer
        importOptions = swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids

        Dim breakLinks As Boolean
        breakLinks = False
        Dim preserveZAxis As Boolean
        preserveZAxis = True

        Dim vResult As Object
        vResult = swAssem.MirrorComponents3(mirrorPlane, compsToInstance, orientations, orientAboutCoM, (compsToMirror),
createDerivedConfigs, filenames, nameModifierType, nameModifier, location, importOptions, breakLinks, preserveZAxis, True)

    End Sub

    Public swApp As SldWorks

```

End Class

Mirror Components II Example (C#)

This example shows how to mirror components in an assembly.

```
//-----
// Preconditions:
// 1. Open C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\advdrawings\98food processor.sldasm.
// 2. Ensure that c:\API\MirrorComps exists.
//
// Postconditions:
// 1. Creates Plane4 mirror plane.
// 2. Creates mirror copies of gear- caddy-1 and middle-gear-1, both aligned to component origins.
// 3. Creates mirror copies of shaft gear-1 and middle-gear plate-1, aligned to PLANE2 and PLANE3, respectively.
// 4. Creates opposite-hand versions of base plate-1 and shaft gear insert-1, called FileName1.sldprt and FileName2.sldprt,
//    respectively, and stored in c:\API\MirrorComps.
// 5. Replaces FileName1.sldprt with check1.sldprt in c:\API\MirrorComps.
// 6. Creates MirrorComponent1 in the FeatureManager design tree.
// 7. Modifies MirrorComponent1 to change the mirror type, the align to origin component orientations, and the align to selection
//    orientations.
//
// NOTE: Because the model is used elsewhere, do not save any changes to it.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace MirrorComponent_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;

        private AssemblyDoc swAssem;
        private Feature swFeat;
        private Feature mirrorPlane;
        private FeatureManager FeatMgr;
        private MirrorComponentFeatureData swMirroCompFeatData;
        private RefPlane myRefPlane;
        private swMirrorComponentOrientation2_e[] compsToMirrorOrientationonOrigin = new swMirrorComponentOrientation2_e[2];
        private swMirrorComponentOrientation2_e[] compsToMirrorOrientationonPlane = new swMirrorComponentOrientation2_e[2];
        private Feature mirrorPlane2;
        private Feature mirrorPlane3;
        private bool[] FlipDir = new bool[2];
        private object flipDirVar;
        private object orientationArray;
        private object orientationSelPlanArray;
        private string[] mirrorCompFileNames = new string[2];
        private string mirrorCompFolder;
        private object namesVar;
        private string[] replaceLocations = new string[2];
        private object replaceLocationsArray;
        private int importOptions;
        private MirrorComponentFeatureData modifyFeatdef;
        private swMirrorComponentOrientation2_e[] compsToMirrorOrientationonOrigin2 = new swMirrorComponentOrientation2_e[2];
        private swMirrorComponentOrientation2_e[] compsToMirrorOrientationonPlane2 = new swMirrorComponentOrientation2_e[2];
        private object orientationArray2;
        private object orientationArrayPlane2;
        private object swCompInst = null;
        private DispatcherWrapper[] CTI = new DispatcherWrapper[2];
        private DispatcherWrapper[] CTSP = new DispatcherWrapper[2];
        private DispatcherWrapper[] CTMOH = new DispatcherWrapper[2];
        private DispatcherWrapper[] ARef = new DispatcherWrapper[2];
        private bool boolstatus;
        private bool retVal;
```

```

public void Main()
{
    swModel = (ModelDoc2)swApp.ActiveDoc;
    FeatMgr = swModel.FeatureManager;

    // Create mirror component feature data object
    swMirroCompFeatData =
(MirrorComponentFeatureData)FeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmMirrorComponent);

    // Create mirror plane
    boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.104250921669188, -0.000236987012272039,
-0.0597199999999418, true, 0, null/* TODO Change to default(_) if this is not a reference type */, 0);
    myRefPlane = (RefPlane)swModel.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0);

    swAssem = (AssemblyDoc)swModel;

    mirrorPlane = (Feature)swAssem.FeatureByName("PLANE4");

    // Specify components to instance align to component origins
    swCompInst = swAssem.GetComponentByName("gear- caddy-1");
    CTI[0] = new Dispatcher(swCompInst);

    swCompInst = swAssem.GetComponentByName("middle-gear-1");
    CTI[1] = new Dispatcher(swCompInst);

    // Specify components to instance align to alignment references
    swCompInst = swAssem.GetComponentByName("shaft gear-1");
    CTSP[0] = new Dispatcher(swCompInst);

    swCompInst = swAssem.GetComponentByName("middle-gear plate-1");
    CTSP[1] = new Dispatcher(swCompInst);

    // Specify components for which to create new opposite-hand versions
    swCompInst = swAssem.GetComponentByName("base plate-1");
    CTMOH[0] = new Dispatcher(swCompInst);

    swCompInst = swAssem.GetComponentByName("shaft gear insert-1");
    CTMOH[1] = new Dispatcher(swCompInst);

    // Specify align to origins component orientations
    compsToMirrorOrientationonOrigin[0] = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredY;
    compsToMirrorOrientationonOrigin[1] = swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredY;
    orientationArray = compsToMirrorOrientationonOrigin;

    // Specify align to selection component orientations
    compsToMirrorOrientationonPlane[0] = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredAndFlippedY;
    compsToMirrorOrientationonPlane[1] =
swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredAndFlippedY;
    orientationSelPlanArray = compsToMirrorOrientationonPlane;

    // Specify the alignment references for the align to selection components
    mirrorPlane2 = (Feature)swAssem.FeatureByName("PLANE2");
    ARef[0] = new Dispatcher(mirrorPlane2);

    mirrorPlane3 = (Feature)swAssem.FeatureByName("PLANE3");
    ARef[1] = new Dispatcher(mirrorPlane3);

    // Specify whether to reverse the direction of alignment
    FlipDir[0] = false;
    FlipDir[1] = false;
    flipDirVar = FlipDir;

    // Specify the opposite-hand version folder
    mirrorCompFolder = @"C:\API\MirrorComps";
}

```

```

// Specify opposite-hand version file names
mirrorCompFileNames[0] = ("FileName1");
mirrorCompFileNames[1] = ("FileName2");
namesVar = mirrorCompFileNames;

// Specify replacement locations for the opposite-hand versions
replaceLocations[0] = (@"C:\API\check1.SLDPRT");
replaceLocations[1] = ("");
replaceLocationsArray = replaceLocations;

// Specify transfer options for the opposite-hand versions
importOptions = (int)swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids;

swMirroCompFeatData.MirrorPlane = mirrorPlane;
swMirroCompFeatData.MirrorType = (int)swMirrorComponentMirrorType_e.swMirrorType_ComponentOrigin;
swMirroCompFeatData.ComponentsToInstanceAlignToComponentOrigin = CTI;
swMirroCompFeatData.ComponentOrientationsAlignToComponentOrigin = orientationArray;
swMirroCompFeatData.ComponentsToInstanceAlignToSelection = CTSP;
swMirroCompFeatData.ComponentOrientationsAlignToSelection = orientationSelPlanArray;
swMirroCompFeatData.AlignmentReferences = ARef;
swMirroCompFeatData.FlipDirections = flipDirVar;
swMirroCompFeatData.SyncFlexibleSubAssemblies = true;
swMirroCompFeatData.OppositeHandComponents = CTMOH;
swMirroCompFeatData.CreateDerivedConfigurations = false;
swMirroCompFeatData.PlaceFilesInOneFolder = true;
swMirroCompFeatData.MirrorComponentsFolderLocation = mirrorCompFolder;
swMirroCompFeatData.MirroredComponentFileNames = namesVar;
swMirroCompFeatData.NameModifierType = (int)swMirrorComponentNameModifier_e.swMirrorComponentName_Custom;
swMirroCompFeatData.ReplaceFileLocations = replaceLocationsArray;
swMirroCompFeatData.MirrorTransferOptions = importOptions;
swMirroCompFeatData.DimXpertScheme = true;
swMirroCompFeatData.BreakLinksToOriginalPart = false;
swMirroCompFeatData.PreserveZAxis = true;
swMirroCompFeatData.PropagateFromOriginalPart = false;

// Create MirrorComponent1
swFeat = FeatMgr.CreateFeature(swMirroCompFeatData);

// Modify MirrorComponent1
modifyFeatdef = (MirrorComponentFeatureData)swFeat.GetDefinition();

// Change mirror type to center of mass
modifyFeatdef.MirrorType = (int)swMirrorComponentMirrorType_e.swMirrorType_CenterOfMass;

// Modify align to origin component orientations
compsToMirrorOrientationonOrigin2[0] =
swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredAndFlippedY;
compsToMirrorOrientationonOrigin2[1] = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredAndFlippedY;
orientationArray2 = compsToMirrorOrientationonOrigin2;
modifyFeatdef.ComponentOrientationsAlignToComponentOrigin = orientationArray2;

// Modify align to selection component orientations
compsToMirrorOrientationonPlane2[0] = swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredY;
compsToMirrorOrientationonPlane2[1] = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredY;
orientationArrayPlane2 = compsToMirrorOrientationonPlane2;
modifyFeatdef.ComponentOrientationsAlignToSelection = orientationArrayPlane2;

retVal = swFeat.ModifyDefinition(modifyFeatdef, swModel, null);
}

public SldWorks swApp;
}
}

```


Mirror Components II Example (VBA)

This example shows how to mirror components in an assembly.

```
'-----
' Preconditions:
' 1. Open C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\advdrawings\98food processor.sldasm.
' 2. Ensure that c:\API\MirrorComps exists.

' Postconditions:
' 1. Creates Plane4 mirror plane.
' 2. Creates mirror copies of gear- caddy-1 and middle-gear-1, both aligned to component origins.
' 3. Creates mirror copies of shaft gear-1 and middle-gear plate-1, aligned to PLANE2 and PLANE3, respectively.
' 4. Creates opposite-hand versions of base plate-1 and shaft gear insert-1, called FileName1.sldprt and FileName2.sldprt,
respectively, and stored in c:\API\MirrorComps.
' 5. Replaces FileName1.sldprt with check1.sldprt in c:\API\MirrorComps.
' 6. Creates MirrorComponent1 in the FeatureManager design tree.
' 7. Modifies MirrorComponent1 to change the mirror type, the align to origin component orientations, and the align to selection
orientations.
' NOTE: Because the model is used elsewhere, do not save any changes to it.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssem As SldWorks.AssemblyDoc
Dim swFeat As SldWorks.Feature
Dim mirrorPlane As SldWorks.Feature
Dim FeatMgr As SldWorks.FeatureManager
Dim swMirroCompFeatData As SldWorks.MirrorComponentFeatureData
Dim myRefPlane As SldWorks.RefPlane
Dim compsToInstance(0 To 3) As Object
Dim compsToSelectionPlane(0 To 1) As Object
Dim compToMirrorOppositeHand(0 To 1) As Object
Dim compsToMirrorOrientationonOrigin(0 To 1) As swMirrorComponentOrientation2_e
Dim compsToMirrorOrientationonPlane(0 To 1) As swMirrorComponentOrientation2_e
Dim mirrorPlane2 As SldWorks.Feature
Dim mirrorPlane3 As SldWorks.Feature
Dim alignmentref(0 To 1) As Object
Dim FlipDir(0 To 1) As Boolean
Dim flipDirVar As Variant
Dim orientationArray As Variant
Dim orientationSelPlanArray As Variant
Dim mirrorCompFileNames(0 To 1) As String
Dim mirrorCompFolder As String
Dim namesVar As Variant
Dim replaceLocations(0 To 1) As String
Dim replaceLocationsArray As Variant
Dim importOptions As Long
Dim modifyFeatdef As MirrorComponentFeatureData
Dim compsToMirrorOrientationonOrigin2(0 To 1) As swMirrorComponentOrientation2_e
Dim compsToMirrorOrientationonPlane2(0 To 1) As swMirrorComponentOrientation2_e
Dim orientationArray2 As Variant
Dim orientationArrayPlane2 As Variant
Dim boolstatus As Boolean
Dim retVal As Boolean

Option Explicit
Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.ActiveDoc
    Set FeatMgr = swModel.FeatureManager

    ' Create mirror component feature data object
```

```

Set swMirroCompFeatData = FeatMgr.CreateDefinition(swFmMirrorComponent)

' Create mirror plane
boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.104250921669188, -2.36987012272039E-04, -5.97199999999418E-02, True,
0, Nothing, 0)
Set myRefPlane = swModel.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0)

Set swAssem = swModel

Set mirrorPlane = swAssem.FeatureByName("PLANE4")

' Specify components to instance align to component origins
Set compsToInstance(0) = swAssem.GetComponentByName("gear- caddy-1")
Set compsToInstance(1) = swAssem.GetComponentByName("middle-gear-1")

' Specify components to instance align to alignment references
Set compsToSelectionPlane(0) = swAssem.GetComponentByName("shaft gear-1")
Set compsToSelectionPlane(1) = swAssem.GetComponentByName("middle-gear plate-1")

' Specify components for which to create new opposite-hand versions
Set compToMirrorOppositeHand(0) = swAssem.GetComponentByName("base plate-1")
Set compToMirrorOppositeHand(1) = swAssem.GetComponentByName("shaft gear insert-1")

' Specify align to origins component orientations
compsToMirrorOrientationonOrigin(0) = swOrientation_MirroredX_MirroredY
compsToMirrorOrientationonOrigin(1) = swOrientation_MirroredAndFlippedX_MirroredY
orientationArray = compsToMirrorOrientationonOrigin

' Specify align to selection component orientations
compsToMirrorOrientationonPlane(0) = swOrientation_MirroredX_MirroredAndFlippedY
compsToMirrorOrientationonPlane(1) = swOrientation_MirroredAndFlippedX_MirroredAndFlippedY
orientationSelPlanArray = compsToMirrorOrientationonPlane

' Specify the alignment references for the align to selection components
Set mirrorPlane2 = swAssem.FeatureByName("PLANE2")
Set mirrorPlane3 = swAssem.FeatureByName("PLANE3")
Set alignmentref(0) = mirrorPlane2
Set alignmentref(1) = mirrorPlane3

' Specify whether to reverse the direction of alignment
FlipDir(0) = False
FlipDir(1) = False
flipDirVar = FlipDir

' Specify the opposite-hand version folder
mirrorCompFolder = "C:\API\MirrorComps"

' Specify opposite-hand version file names
mirrorCompFileNames(0) = ("FileName1")
mirrorCompFileNames(1) = ("FileName2")
namesVar = mirrorCompFileNames

' Specify replacement locations for the opposite-hand versions
replaceLocations(0) = ("C:\API\check1.SLDPRT")
replaceLocations(1) = ("")
replaceLocationsArray = replaceLocations

' Specify transfer options for the opposite-hand versions
importOptions = swMirrorPartOptions_ImportSolids

swMirroCompFeatData.mirrorPlane = mirrorPlane
swMirroCompFeatData.MirrorType = swMirrorType_ComponentOrigin
swMirroCompFeatData.ComponentsToInstanceAlignToComponentOrigin = compsToInstance
swMirroCompFeatData.ComponentOrientationsAlignToComponentOrigin = orientationArray
swMirroCompFeatData.ComponentsToInstanceAlignToSelection = compsToSelectionPlane

```

```

swMirroCompFeatData.ComponentOrientationsAlignToSelection = orientationSelPlanArray
swMirroCompFeatData.AlignmentReferences = alignmentref
swMirroCompFeatData.FlipDirections = flipDirVar
swMirroCompFeatData.SyncFlexibleSubAssemblies = True
swMirroCompFeatData.OppositeHandComponents = compToMirrorOppositeHand
swMirroCompFeatData.CreateDerivedConfigurations = False
swMirroCompFeatData.PlaceFilesInOneFolder = True
swMirroCompFeatData.MirrorComponentsFolderLocation = mirrorCompFolder
swMirroCompFeatData.MirroredComponentFilenames = namesVar
swMirroCompFeatData.nameModifierType = swMirrorComponentName_Custom
swMirroCompFeatData.ReplaceFileLocations = replaceLocationsArray
swMirroCompFeatData.MirrorTransferOptions = importOptions
swMirroCompFeatData.DimXpertScheme = True
swMirroCompFeatData.BreakLinksToOriginalPart = False
swMirroCompFeatData.preserveZAxis = True
swMirroCompFeatData.PropagateFromOriginalPart = False

' Create MirrorComponent1
Set swFeat = FeatMgr.CreateFeature(swMirroCompFeatData)

' Modify MirrorComponent1
Set modifyFeatdef = swFeat.GetDefinition

' Change mirror type to center of mass
modifyFeatdef.MirrorType = swMirrorType_CenterOfMass

' Modify align to origin component orientations
compsToMirrorOrientationonOrigin2(0) = swOrientation_MirroredAndFlippedX_MirroredAndFlippedY
compsToMirrorOrientationonOrigin2(1) = swOrientation_MirroredX_MirroredAndFlippedY
orientationArray2 = compsToMirrorOrientationonOrigin2
modifyFeatdef.ComponentOrientationsAlignToComponentOrigin = orientationArray2

' Modify align to selection component orientations
compsToMirrorOrientationonPlane2(0) = swOrientation_MirroredAndFlippedX_MirroredY
compsToMirrorOrientationonPlane2(1) = swOrientation_MirroredX_MirroredY
orientationArrayPlane2 = compsToMirrorOrientationonPlane2
modifyFeatdef.ComponentOrientationsAlignToSelection = orientationArrayPlane2

retVal = swFeat.ModifyDefinition(modifyFeatdef, swModel, Nothing)

```

End Sub

Mirror Components II Example (VB.NET)

This example shows how to mirror components in an assembly.

```
'-----
' Preconditions:
' 1. Open C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\advdrawings\98food processor.sldasm.
' 2. Ensure that c:\API\MirrorComps exists.

' Postconditions:
' 1. Creates Plane4 mirror plane.
' 2. Creates mirror copies of gear- caddy-1 and middle-gear-1, both aligned to component origins.
' 3. Creates mirror copies of shaft gear-1 and middle-gear plate-1, aligned to PLANE2 and PLANE3, respectively.
' 4. Creates opposite-hand versions of base plate-1 and shaft gear insert-1, called FileName1.sldprt and FileName2.sldprt,
respectively, and stored in c:\API\MirrorComps.
' 5. Replaces FileName1.sldprt with check1.sldprt in c:\API\MirrorComps.
' 6. Creates MirrorComponent1 in the FeatureManager design tree.
' 7. Modifies MirrorComponent1 to change the mirror type, the align to origin component orientations, and the align to selection
orientations.
'
' NOTE: Because the model is used elsewhere, do not save any changes to it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swAssem As AssemblyDoc
    Dim swFeat As Feature
    Dim mirrorPlane As Feature
    Dim FeatMgr As FeatureManager
    Dim swMirroCompFeatData As MirrorComponentFeatureData
    Dim myRefPlane As RefPlane
    Dim compsToMirrorOrientationonOrigin(1) As swMirrorComponentOrientation2_e
    Dim compsToMirrorOrientationonPlane(1) As swMirrorComponentOrientation2_e
    Dim mirrorPlane2 As Feature
    Dim mirrorPlane3 As Feature
    Dim FlipDir(1) As Boolean
    Dim flipDirVar As Object
    Dim orientationArray As Object
    Dim orientationSelPlanArray As Object
    Dim mirrorCompFileNames(1) As String
    Dim mirrorCompFolder As String
    Dim namesVar As Object
    Dim replaceLocations(1) As String
    Dim replaceLocationsArray As Object
    Dim importOptions As Integer
    Dim modifyFeatdef As MirrorComponentFeatureData
    Dim compsToMirrorOrientationonOrigin2(1) As swMirrorComponentOrientation2_e
    Dim compsToMirrorOrientationonPlane2(1) As swMirrorComponentOrientation2_e
    Dim orientationArray2 As Object
    Dim orientationArrayPlane2 As Object
    Dim swCompInst As Object = Nothing
    Dim CTI(1) As DispatcherWrapper
    Dim CTSP(1) As DispatcherWrapper
    Dim CTMOH(1) As DispatcherWrapper
    Dim ARef(1) As DispatcherWrapper
```

```

Dim boolstatus As Boolean
Dim retVal As Boolean

Sub main()

    swModel = swApp.ActiveDoc
    FeatMgr = swModel.FeatureManager

    ' Create mirror component feature data object
    swMirroCompFeatData = FeatMgr.CreateDefinition(swFeatureNameID_e.swFmMirrorComponent)

    ' Create mirror plane
    boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.104250921669188, -0.000236987012272039, -0.05971999999999418, True,
0, Nothing, 0)
    myRefPlane = swModel.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0)

    swAssem = swModel

    mirrorPlane = swAssem.FeatureByName("PLANE4")

    ' Specify components to instance align to component origins
    swCompInst = swAssem.GetComponentByName("gear- caddy-1")
    CTI(0) = New DispatchWrapper(swCompInst)

    swCompInst = swAssem.GetComponentByName("middle-gear-1")
    CTI(1) = New DispatchWrapper(swCompInst)

    ' Specify components to instance align to alignment references
    swCompInst = swAssem.GetComponentByName("shaft gear-1")
    CTSP(0) = New DispatchWrapper(swCompInst)

    swCompInst = swAssem.GetComponentByName("middle-gear plate-1")
    CTSP(1) = New DispatchWrapper(swCompInst)

    ' Specify components for which to create new opposite-hand versions
    swCompInst = swAssem.GetComponentByName("base plate-1")
    CTMOH(0) = New DispatchWrapper(swCompInst)

    swCompInst = swAssem.GetComponentByName("shaft gear insert-1")
    CTMOH(1) = New DispatchWrapper(swCompInst)

    ' Specify align to origins component orientations
    compsToMirrorOrientationonOrigin(0) = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredY
    compsToMirrorOrientationonOrigin(1) = swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredY
    orientationArray = compsToMirrorOrientationonOrigin

    ' Specify align to selection component orientations
    compsToMirrorOrientationonPlane(0) = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredAndFlippedY
    compsToMirrorOrientationonPlane(1) = swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredAndFlippedY
    orientationSelPlanArray = compsToMirrorOrientationonPlane

    ' Specify the alignment references for the align to selection components
    mirrorPlane2 = swAssem.FeatureByName("PLANE2")
    ARef(0) = New DispatchWrapper(mirrorPlane2)

    mirrorPlane3 = swAssem.FeatureByName("PLANE3")
    ARef(1) = New DispatchWrapper(mirrorPlane3)

    ' Specify whether to reverse the direction of alignment
    FlipDir(0) = False
    FlipDir(1) = False
    flipDirVar = FlipDir

    ' Specify the opposite-hand version folder

```

```

mirrorCompFolder = "C:\API\MirrorComps"

' Specify opposite-hand version file names
mirrorCompFileNames(0) = ("FileName1")
mirrorCompFileNames(1) = ("FileName2")
namesVar = mirrorCompFileNames

' Specify replacement locations for the opposite-hand versions
replaceLocations(0) = ("C:\API\check1.SLDPR1")
replaceLocations(1) = ("")
replaceLocationsArray = replaceLocations

' Specify transfer options for the opposite-hand versions
importOptions = swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids

swMirroCompFeatData.mirrorPlane = mirrorPlane
swMirroCompFeatData.MirrorType = swMirrorComponentMirrorType_e.swMirrorType_ComponentOrigin
swMirroCompFeatData.ComponentsToInstanceAlignToComponentOrigin = CTI
swMirroCompFeatData.ComponentOrientationsAlignToComponentOrigin = orientationArray
swMirroCompFeatData.ComponentsToInstanceAlignToSelection = CTSP
swMirroCompFeatData.ComponentOrientationsAlignToSelection = orientationSelPlanArray
swMirroCompFeatData.AlignmentReferences = ARef
swMirroCompFeatData.FlipDirections = flipDirVar
swMirroCompFeatData.SyncFlexibleSubAssemblies = True
swMirroCompFeatData.OppositeHandComponents = CTMOH
swMirroCompFeatData.CreateDerivedConfigurations = False
swMirroCompFeatData.PlaceFilesInOneFolder = True
swMirroCompFeatData.MirrorComponentsFolderLocation = mirrorCompFolder
swMirroCompFeatData.MirroredComponentFileNames = namesVar
swMirroCompFeatData.MirroredComponentFileNames = mirrorCompFileNames
swMirroCompFeatData.NameModifierType = swMirrorComponentNameModifier_e.swMirrorComponentName_Custom
swMirroCompFeatData.ReplaceFileLocations = replaceLocationsArray
swMirroCompFeatData.MirrorTransferOptions = importOptions
swMirroCompFeatData.DimXpertScheme = True
swMirroCompFeatData.BreakLinksToOriginalPart = False
swMirroCompFeatData.preserveZAxis = True
swMirroCompFeatData.PropagateFromOriginalPart = False

' Create MirrorComponent1
swFeat = FeatMgr.CreateFeature(swMirroCompFeatData)

' Modify MirrorComponent1
modifyFeatdef = swFeat.GetDefinition

' Change mirror type to center of mass
modifyFeatdef.MirrorType = swMirrorComponentMirrorType_e.swMirrorType_CenterOfMass

' Modify align to origin component orientations
compsToMirrorOrientationonOrigin2(0) =
swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredAndFlippedY
compsToMirrorOrientationonOrigin2(1) = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredAndFlippedY
orientationArray2 = compsToMirrorOrientationonOrigin2
modifyFeatdef.ComponentOrientationsAlignToComponentOrigin = orientationArray2

' Modify align to selection component orientations
compsToMirrorOrientationonPlane2(0) = swMirrorComponentOrientation2_e.swOrientation_MirroredAndFlippedX_MirroredY
compsToMirrorOrientationonPlane2(1) = swMirrorComponentOrientation2_e.swOrientation_MirroredX_MirroredY
orientationArrayPlane2 = compsToMirrorOrientationonPlane2
modifyFeatdef.ComponentOrientationsAlignToSelection = orientationArrayPlane2

retVal = swFeat.ModifyDefinition(modifyFeatdef, swModel, Nothing)

```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>
```

```
Public swApp As SldWorks  
End Class
```

Mirror Sheet-metal Part Example (C#)

This example shows how to mirror a sheet-metal part.

```
//-----
// Preconditions:
// 1. Verify that the sheet-metal part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified sheet-metal part.
// 2. Creates a reference plane about which to mirror the
//    sheet-metal part.
// 3. Creates a new part document containing the mirrored
//    sheet-metal part, which includes the sheet-metal
//    information in the mirrored part.
// 4. Examine the graphics area and the Immediate window.
//
// NOTE: Because this part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace MirrorSheetMetalPartCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            PartDoc swPart = default(PartDoc);
            Feature swMirrorFeature = default(Feature);
            Feature swFeature = default(Feature);
            ModelDoc2 swResultPart = null;
            MirrorPartFeatureData swMirrorFeatData = default(MirrorPartFeatureData);
            RefPlane swRefPlane = default(RefPlane);
            Entity swPlane = default(Entity);
            int mirrorOptions = 0;
            int mirrorType = 0;
            swSelectType_e selType = default(swSelectType_e);
            string filename = null;
            int errors = 0;
            bool status = false;
            int warnings = 0;

            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(filename, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            if (swModel == null)
                return;
            if (swModel.GetType() != (int)swDocumentTypes_e.swDocPART)
                return;

            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swRefPlane = (RefPlane)swFeatureMgr.InsertRefPlane(8, 0.09, 0, 0, 0, 0);
            status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            selType = (swSelectType_e)swSelMgr.GetSelectedObjectType3(1, -1);
            if (!(selType == swSelectType_e.swSelDATUMPLANES))
                return;

            swPart = (PartDoc)swModel;
            mirrorOptions = (int)swMirrorPartOptions_e.swMirrorPartOptions_ImportSMInfo + (int)swMirrorPartOptions_e.swMirrorPartOptions_ImportIndProps +
(int)swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids + (int)swMirrorPartOptions_e.swMirrorPartOptions_ImportCutListProperties;
            swMirrorFeature = (Feature)swPart.MirrorPart2(false, mirrorOptions, out swResultPart);
            if (swMirrorFeature == null)
            {
                Debug.Print("No feature!");
            }
            else
            {
                Debug.Print("Feature: " + swMirrorFeature.Name);
            }

            if (swResultPart == null)
            {
                Debug.Print("No new part! ");
            }
            else
            {
                Debug.Print("Part document title: " + swResultPart.GetTitle());
            }

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swMirrorFeature.Select2(false, -1);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swMirrorFeatData = (MirrorPartFeatureData)swFeature.GetDefinition();

            swMirrorFeatData.AccessSelections(swModel, null);

            Debug.Print(" Path name = " + swMirrorFeatData.PathName);
            Debug.Print(" Import: ");
            Debug.Print(" Solid bodies? " + swMirrorFeatData.SolidBodies);
            Debug.Print(" Cut-list properties? " + swMirrorFeatData.CutListProperties);
            Debug.Print(" Sheet-metal information? " + swMirrorFeatData.SheetMetalInformation);
            Debug.Print(" Unlocked properties? " + swMirrorFeatData.UnlockedProperties);

            mirrorType = swMirrorFeatData.GetMirrorPlaneType();
        }
    }
}
```

```

        Debug.Print(" Mirror plane type as defined in swMirrorPlaneType_e = " + mirrorType);
        swRefPlane = (RefPlane)swMirrorFeatData.GetMirrorPlane();
        swMirrorFeatData.ReleaseSelectionAccess();
        swPlane = (Entity)swRefPlane;
        swPlane.Select(false);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}

```

Mirror Sheet-metal Part Example (VBA)

This example shows how to mirror a sheet-metal part.

```
'-----
' Preconditions:
' 1. Verify that the sheet-metal part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified sheet-metal part.
' 2. Creates a reference plane about which to mirror the
'    sheet-metal part.
' 3. Creates a new part document containing the mirrored
'    sheet-metal part, which includes the sheet-metal
'    information in the mirrored part.
' 4. Examine the graphics area and the Immediate window.
'
' NOTE: Because this part document is used elsewhere, do
' not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swSelMgr As SldWorks.SelectionMgr
Dim swPart As SldWorks.PartDoc
Dim swMirrorFeature As SldWorks.Feature
Dim swFeature As SldWorks.Feature
Dim swResultPart As SldWorks.ModelDoc2
Dim swMirrorFeatData As SldWorks.MirrorPartFeatureData
Dim swRefPlane As SldWorks.refPlane
Dim swPlane As SldWorks.Entity
Dim mirrorOptions As Long
Dim mirrorType As Long
Dim selType As swSelectType_e
Dim filename As String
Dim errors As Long
Dim status As Boolean
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
    Set swModel = swApp.OpenDoc6(filename, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    If swModel Is Nothing Then Exit Sub
    If swModel.GetType <> swDocPART Then Exit Sub

    Set swModelDocExt = swModel.Extension
    Set swFeatureMgr = swModel.FeatureManager
    status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swRefPlane = swFeatureMgr.InsertRefPlane(8, 0.09, 0, 0, 0, 0)
    status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    selType = swSelMgr.GetSelectedObjectType3(1, -1)
    If Not (selType = swSelDATUMPLANES) Then Exit Sub

    Set swPart = swModel
    mirrorOptions = swMirrorPartOptions_ImportSMInfo + swMirrorPartOptions_ImportIndProps + swMirrorPartOptions_ImportSolids +
swMirrorPartOptions_ImportCutListProperties
    Set swMirrorFeature = swPart.MirrorPart2(False, mirrorOptions, swResultPart)
    If swMirrorFeature Is Nothing Then
        Debug.Print "No feature!"
    Else
        Debug.Print "Feature: " & swMirrorFeature.Name
    End If

    If swResultPart Is Nothing Then
        Debug.Print "No new part! "
    Else
        Debug.Print "Part document title: " & swResultPart.GetTitle
    End If

    Set swModel = swApp.ActiveDoc
    swMirrorFeature.Select2 False, -1
    Set swSelMgr = swModel.SelectionManager
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swMirrorFeatData = swFeature.GetDefinition

    swMirrorFeatData.AccessSelections swModel, Nothing

    Debug.Print " Path name = " & swMirrorFeatData.PathName
    Debug.Print " Import: "
    Debug.Print " Solid bodies? " & swMirrorFeatData.SolidBodies
    Debug.Print " Cut-list properties? " & swMirrorFeatData.CutListProperties
    Debug.Print " Sheet-metal information? " & swMirrorFeatData.SheetMetalInformation
    Debug.Print " Unlocked properties? " & swMirrorFeatData.UnlockedProperties

    mirrorType = swMirrorFeatData.GetMirrorPlaneType
    Debug.Print " Mirror plane type as defined in swMirrorPlaneType_e = " & mirrorType
    Set swRefPlane = swMirrorFeatData.GetMirrorPlane
```

```
swMirrorFeatData.ReleaseSelectionAccess  
Set swPlane = swRefPlane  
swPlane.Select False
```

```
End Sub
```

Mirror Sheet-metal Part Example (VB.NET)

This example shows how to mirror a sheet-metal part.

```

-----
Preconditions:
1. Verify that the sheet-metal part to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified sheet-metal part.
2. Creates a reference plane about which to mirror the sheet-metal part.
3. Creates a new part document containing the mirrored sheet-metal part, which includes the sheet-metal information in the mirrored part.
4. Examine the graphics area and the Immediate window.
NOTE: Because this part document is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureMgr As FeatureManager
        Dim swSelMgr As SelectionMgr
        Dim swPart As PartDoc
        Dim swMirrorFeature As Feature
        Dim swFeature As Feature
        Dim swResultPart As ModelDoc2 = Nothing
        Dim swMirrorFeatData As MirrorPartFeatureData
        Dim swRefPlane As RefPlane
        Dim swPlane As Entity
        Dim mirrorOptions As Integer
        Dim mirrorType As Integer
        Dim selType As swSelectType_e
        Dim filename As String
        Dim errors As Integer
        Dim status As Boolean
        Dim warnings As Integer

        filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\2012-sm.sldprt"
        swModel = swApp.OpenDoc6(filename, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        If swModel Is Nothing Then Exit Sub
        If swModel.GetType <> swDocumentTypes_e.swDocPART Then Exit Sub

        swModelDocExt = swModel.Extension
        swFeatureMgr = swModel.FeatureManager
        status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swRefPlane = swFeatureMgr.InsertRefPlane(8, 0.09, 0, 0, 0, 0)
        status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        selType = swSelMgr.GetSelectedObjectType3(1, -1)
        If Not (selType = swSelectType_e.swSelDATUMPLANES) Then Exit Sub

        swPart = swModel
        mirrorOptions = swMirrorPartOptions_e.swMirrorPartOptions_ImportSMInfo + swMirrorPartOptions_e.swMirrorPartOptions_ImportIndProps + swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids + swMirrorPartOptions_e.swMirrorPartOptions_ImportCutListProperties
        swMirrorFeature = swPart.MirrorPart2(False, mirrorOptions, swResultPart)
        If swMirrorFeature Is Nothing Then
            Debug.Print("No feature!")
        Else
            Debug.Print("Feature: " & swMirrorFeature.Name)
        End If

        If swResultPart Is Nothing Then
            Debug.Print("No new part! ")
        Else
            Debug.Print("Part document title: " & swResultPart.GetTitle)
        End If

        swModel = swApp.ActiveDoc
        swMirrorFeature.Select2(False, -1)
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swMirrorFeatData = swFeature.GetDefinition

        swMirrorFeatData.AccessSelections(swModel, Nothing)

        Debug.Print(" Path name = " & swMirrorFeatData.PathName)
        Debug.Print(" Import: ")
        Debug.Print(" Solid bodies? " & swMirrorFeatData.SolidBodies)
        Debug.Print(" Cut-list properties? " & swMirrorFeatData.CutListProperties)
        Debug.Print(" Sheet-metal information? " & swMirrorFeatData.SheetMetalInformation)
        Debug.Print(" Unlocked properties? " & swMirrorFeatData.UnlockedProperties)

        mirrorType = swMirrorFeatData.GetMirrorPlaneType
        Debug.Print(" Mirror plane type as defined in swMirrorPlaneType_e = " & mirrorType)
        swRefPlane = swMirrorFeatData.GetMirrorPlane
        swMirrorFeatData.ReleaseSelectionAccess()
        swPlane = swRefPlane
        swPlane.Select(False)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As SldWorks

End Class

Mirror View Example (C#)

This example shows how to mirror a view.

```
//-----
// Preconditions:
// 1. Verify that the drawing to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the drawing.
// 2. Mirrors the drawing view.
// 3. Examine the drawing and Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc swDrawing = default(DrawingDoc);
            View swView = default(View);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool mirrored = false;
            int orientation = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\clamp1.slddrw";
            swDrawing = (DrawingDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
errors, ref warnings);
            swView = (View)swDrawing.GetFirstView(); //This is the drawing template
            swView = (View)swView.GetNextView(); //This is the first drawing view in the drawing
            swView.SetMirrorViewOrientation(true, (int)swMirrorViewPositions_e.swMirrorViewPosition_Horizontal);
            swView.GetMirrorViewOrientation(out mirrored, out orientation);
            Debug.Print("Mirrored? " + mirrored);
            Debug.Print("Orientation (0 = horizontal)? " + orientation);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Mirror View Example (VBA)

This example shows how to mirror a view.

```
'-----  
' Preconditions:  
' 1. Verify that the drawing to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the drawing.  
' 2. Mirrors the drawing view.  
' 3. Examine the drawing and Immediate window.  
'  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swView As SldWorks.View  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
Dim mirrored As Boolean  
Dim orientation As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.slddrw"  
    Set swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swView = swDrawing.GetFirstView 'This is the drawing template  
    Set swView = swView.GetNextView 'This is the first drawing view in the drawing  
    swView.SetMirrorViewOrientation True, swMirrorViewPositions_e.swMirrorViewPosition_Horizontal  
    swView.GetMirrorViewOrientation mirrored, orientation  
    Debug.Print "Mirrored? " & mirrored  
    Debug.Print "Orientation (0 = horizontal)? " & orientation  
  
End Sub
```

Mirror View Example (VB.NET)

This example shows how to mirror a view.

```
'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Mirrors the drawing view.
' 3. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swDrawing As DrawingDoc
        Dim swView As View
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim mirrored As Boolean
        Dim orientation As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.slddrw"
        swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swView = swDrawing.GetFirstView 'This is the drawing template
        swView = swView.GetNextView 'This is the first drawing view in the drawing
        swView.SetMirrorViewOrientation(True, swMirrorViewPositions_e.swMirrorViewPosition_Horizontal)
        swView.GetMirrorViewOrientation(mirrored, orientation)
        Debug.Print("Mirrored? " & mirrored)
        Debug.Print("Orientation (0 = horizontal)? " & orientation)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Modify and Reload Sheet Format Template Example (C#)

This example shows how to modify and reload a sheet format template.

```
//-----
// Preconditions:
// 1. Make a copy of:
//    C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
// 2. Create a new blank drawing using standard sheet size A0 (ISO).
// 3. Add another blank sheet to the drawing, for a total of two sheets.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Modifies the sheet format template to include a new
//    note.
// 2. Examine Sheet1, Sheet2, and the Immediate window.
// 3. Delete:
//    C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
// 4. Rename the copy that you made in Preconditions step 1 to:
//    C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        const bool TEST_APPLY_CHANGES_TO_ALL = true;

        private string GetReloadResult(swReloadTemplateResult_e result)
        {
            string functionReturnValue = null;
            switch (result)
            {
                case (swReloadTemplateResult_e)swReloadTemplateResult_e.swReloadTemplate_Success:
                    functionReturnValue = "Success";
                    break;
                case (swReloadTemplateResult_e)swReloadTemplateResult_e.swReloadTemplate_UnknownError:
                    functionReturnValue = "FAIL - Unknown Error";
                    break;
                case (swReloadTemplateResult_e)swReloadTemplateResult_e.swReloadTemplate_FileNotFound:
                    functionReturnValue = "FAIL - File Not Found";
                    break;
                case (swReloadTemplateResult_e)swReloadTemplateResult_e.swReloadTemplate_CustomSheet:
                    functionReturnValue = "FAIL - Custom Sheet";
                    break;
                case (swReloadTemplateResult_e)swReloadTemplateResult_e.swReloadTemplate_ViewOnly:
                    functionReturnValue = "FAIL - View Only";
                    break;
                default:
                    functionReturnValue = "FAIL - <unrecognized error code - " + result + ">";
                    break;
            }
            return functionReturnValue;
        }

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            if (swModel == null)
            {
                Debug.Print("Create a new empty drawing and add a second sheet to the drawing.");
                return;
            }

            if (swModel.GetType() != (int)swDocumentTypes_e.swDocDRAWING)
                return;

            DrawingDoc swDrwng = default(DrawingDoc);
            swDrwng = (DrawingDoc)swModel;

            //Get the current sheet
            Sheet activeSheet = default(Sheet);
            activeSheet = (Sheet)swDrwng.GetCurrentSheet();
            Debug.Print("Active sheet name: " + activeSheet.GetName());

            //Get the sheet format template
            string templateName = null;
            templateName = activeSheet.GetTemplateName();
            Debug.Print("Sheet format template name to modify: " + templateName);
            swDrwng.EditTemplate();

            //Add a new note to the sheet format template
            Note swNote = default(Note);
            swNote = (Note)swModel.InsertNote("A New Note");
            Annotation swAnno = default(Annotation);
            swAnno = (Annotation)swNote.GetAnnotation();
            swAnno.SetPosition2(0, 0.2, 0);
            TextFormat txtFormat = default(TextFormat);
            txtFormat = (TextFormat)swAnno.GetTextFormat(0);
            txtFormat.Backwards = (txtFormat.Backwards == false);
            txtFormat.Bold = true;
            txtFormat.CharHeightInPts = 10 * txtFormat.CharHeightInPts;
            swAnno.SetTextFormat(0, false, txtFormat);
        }
    }
}
```

```

swDrwng.EditSheet();

//At this point, the active sheet's format has changed

if (TEST_APPLY_CHANGES_TO_ALL)
{
    //Save sheet format back to original sheet format template
    activeSheet.SaveFormat(templateName);
    //Reload all other sheets from the updated sheet format template
    object[] vSheetNames = null;
    vSheetNames = (object[])swDrwng.GetSheetNames();
    foreach (string vName in vSheetNames)
    {
        if (vName != (string)activeSheet.GetName())
        {
            Debug.Print("Other sheet name: " + vName);
            Sheet otherSheet = default(Sheet);
            otherSheet = (Sheet)swDrwng.get_Sheet(vName);
            if (otherSheet.GetTemplateName() == templateName)
            {
                swReloadTemplateResult_e reloadResult = default(swReloadTemplateResult_e);

                //Keep modifications and and reload all other elements
                //from original sheet format template
                reloadResult = (swReloadTemplateResult_e)otherSheet.ReloadTemplate(true);

                //Discard modifications and reload all elements from
                //original sheet format template
                //reloadResult = otherSheet.ReloadTemplate(False)

                Debug.Print("Reload sheet format for <" + otherSheet.GetName() + ">: " + GetReloadResult(reloadResult));
            }
        }
    }
    swDrwng.ActivateSheet(activeSheet.GetName());
}
else
{
    //Discard the changes and reload the original sheet format template
    swReloadTemplateResult_e reloadResult = default(swReloadTemplateResult_e);
    reloadResult = (swReloadTemplateResult_e)activeSheet.ReloadTemplate(false);
    Debug.Print("Done - " + GetReloadResult(reloadResult));
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Modify and Reload Sheet Format Template Example (VBA)

This example shows how to modify and reload a sheet format template.

```
'-----
' Preconditions:
' 1. Make a copy of:
'    C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
' 2. Create a new blank drawing using standard sheet size A0 (ISO).
' 3. Add another blank sheet to the drawing, for a total of two sheets.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Modifies the sheet format template to include a new
'    note.
' 2. Examine Sheet1, Sheet2, and the Immediate window.
' 3. Delete:
'    C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
' 4. Rename the copy that you made in Preconditions step 1 to:
'    C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
'-----
```

Option Explicit

Const TEST_APPLY_CHANGES_TO_ALL = True

Sub main()

Dim swApp As SldWorks.SldWorks
Set swApp = Application.SldWorks

Dim swModel As ModelDoc2
Set swModel = swApp.ActiveDoc

If swModel Is Nothing Then
 Debug.Print "Create a new empty drawing and add a second sheet to the drawing."
 Exit Sub
End If

If swModel.GetType <> swDocDRAWING Then Exit Sub

Dim swDrwng As DrawingDoc
Set swDrwng = swModel

'Get the current sheet
Dim activeSheet As Sheet
Set activeSheet = swDrwng.GetCurrentSheet

Debug.Print "Active sheet name: " & activeSheet.GetName

'Get the sheet format template
Dim templateName As String
templateName = activeSheet.GetTemplateName()
Debug.Print "Sheet format template name to modify: " & templateName

swDrwng.EditTemplate

'Add a new note to the sheet format template
Dim swNote As Note
Set swNote = swModel.InsertNote("A New Note")
Dim swAnno As Annotation
Set swAnno = swNote.GetAnnotation
swAnno.SetPosition2 0, 0.2, 0
Dim txtFormat As TextFormat
Set txtFormat = swAnno.GetTextFormat(0)
txtFormat.Backwards = (txtFormat.Backwards = False)
txtFormat.Bold = True
txtFormat.CharHeightInPts = 10 * txtFormat.CharHeightInPts
swAnno.SetTextFormat 0, False, txtFormat

swDrwng.EditSheet

'At this point, the active sheet's format has changed

If TEST_APPLY_CHANGES_TO_ALL Then

'Save sheet format back to original sheet format template
activeSheet.SaveFormat templateName

'Reload all other sheets from the updated sheet format template
Dim vSheetNames As Variant
vSheetNames = swDrwng.GetSheetNames()
Dim vName As Variant
For Each vName In vSheetNames
 If Not vName = activeSheet.GetName Then
 Debug.Print "Other sheet name: " & vName
 Dim otherSheet As Sheet
 Set otherSheet = swDrwng.Sheet(vName)

 If otherSheet.GetTemplateName = templateName Then
 Dim reloadResult As swReloadTemplateResult_e

 'Keep modifications and and reload all other elements
 'from original sheet format template
 reloadResult = otherSheet.ReloadTemplate(True)

 'Discard modifications and reload all elements from
 'original sheet format template
 'reloadResult = otherSheet.ReloadTemplate(False)

```

        Debug.Print "Reload sheet format for <" & otherSheet.GetName & ">: " & GetReloadResult(reloadResult)
    End If
End If

Next

swDrwng.ActiveSheet = activeSheet.GetName

Else

'Discard the changes and reload the original sheet format template
reloadResult = activeSheet.ReloadTemplate(False)
Debug.Print "Done - " & GetReloadResult(reloadResult)

End If

End Sub

Private Function GetReloadResult(ByVal result As swReloadTemplateResult_e) As String
    Select Case result
    Case swReloadTemplate_Success
        GetReloadResult = "Success"
    Case swReloadTemplate_UnknownError
        GetReloadResult = "FAIL - Unknown Error"
    Case swReloadTemplate_FileNotFound
        GetReloadResult = "FAIL - File Not Found"
    Case swReloadTemplate_CustomSheet
        GetReloadResult = "FAIL - Custom Sheet"
    Case swReloadTemplate_ViewOnly
        GetReloadResult = "FAIL - View Only"
    Case Else
        GetReloadResult = "FAIL - <unrecognized error code - " & result & ">"
    End Select
End Function

```

Modify and Reload Sheet Format Template Example (VB.NET)

This example shows how to modify and reload a sheet format template.

```

-----
Preconditions:
1. Make a copy of:
   C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
2. Create a new blank drawing using standard sheet size A0 (ISO).
3. Add another blank sheet to the drawing, for a total of two sheets.
4. Open the Immediate window.
Postconditions:
1. Modifies the sheet format template to include a new
   note.
2. Examine Sheet1, Sheet2, and the Immediate window.
3. Delete:
   C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
4. Rename the copy that you made in Preconditions step 1 to:
   C:\ProgramData\SolidWorks\SOLIDWORKS version\lang\english\sheetformat\A0 - iso.slddrt.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Const TEST_APPLY_CHANGES_TO_ALL As Boolean = True

    Private Function GetReloadResult(ByVal result As swReloadTemplateResult_e) As String
        Select Case result
            Case swReloadTemplateResult_e.swReloadTemplate_Success
                GetReloadResult = "Success"
            Case swReloadTemplateResult_e.swReloadTemplate_UnknownError
                GetReloadResult = "FAIL - Unknown Error"
            Case swReloadTemplateResult_e.swReloadTemplate_FileNotFound
                GetReloadResult = "FAIL - File Not Found"
            Case swReloadTemplateResult_e.swReloadTemplate_CustomSheet
                GetReloadResult = "FAIL - Custom Sheet"
            Case swReloadTemplateResult_e.swReloadTemplate_ViewOnly
                GetReloadResult = "FAIL - View Only"
            Case Else
                GetReloadResult = "FAIL - <unrecognized error code - " & result & ">"
        End Select
    End Function

    Public Sub main()

        Dim swModel As ModelDoc2
        swModel = swApp.ActiveDoc

        If swModel Is Nothing Then
            Debug.Print("Create a new empty drawing and add a second sheet to the drawing.")
            Exit Sub
        End If

        If swModel.GetType <> swDocumentTypes_e.swDocDRAWING Then Exit Sub

        Dim swDrwng As DrawingDoc
        swDrwng = swModel

        'Get the current sheet
        Dim activeSheet As Sheet
        activeSheet = swDrwng.GetCurrentSheet
        Debug.Print("Active sheet name: " & activeSheet.GetName)

        'Get the sheet format template
        Dim templateName As String
        templateName = activeSheet.GetTemplateName()
        Debug.Print("Sheet format template name to modify: " & templateName)
        swDrwng.EditTemplate()

        'Add a new note to the sheet format template
        Dim swNote As Note
        swNote = swModel.InsertNote("A New Note")
        Dim swAnno As Annotation
        swAnno = swNote.GetAnnotation
        swAnno.SetPosition2(0, 0.2, 0)
        Dim txtFormat As TextFormat
        txtFormat = swAnno.GetTextFormat(0)
        txtFormat.Backwards = (txtFormat.Backwards = False)
        txtFormat.Bold = True
        txtFormat.CharHeightInPts = 10 * txtFormat.CharHeightInPts
        swAnno.SetTextFormat(0, False, txtFormat)
        swDrwng.EditSheet()

        'At this point, the active sheet's format has changed

        If TEST_APPLY_CHANGES_TO_ALL Then
            'Save sheet format back to original sheet format template
            activeSheet.SaveFormat(templateName)
            'Reload all other sheets from the updated sheet format template
            Dim vSheetNames As Object
            vSheetNames = swDrwng.GetSheetNames()
            Dim vName As Object
            For Each vName In vSheetNames
                If Not vName = activeSheet.GetName Then
                    Debug.Print("Other sheet name: " & vName)
                    Dim otherSheet As Sheet
                    otherSheet = swDrwng.Sheet(vName)
                    If otherSheet.GetTemplateName = templateName Then
                        Dim reloadResult As swReloadTemplateResult_e

                        'Keep modifications and and reload all other elements
                        'from original sheet format template
                        reloadResult = otherSheet.ReloadTemplate(True)

                        'Discard modifications and reload all elements from

```

```

        'original sheet format template
        'reloadResult = otherSheet.ReloadTemplate(False)

        Debug.Print("Reload sheet format for <" & otherSheet.GetName & ">: " & GetReloadResult(reloadResult))

    End If
End If
Next
swDrwng.ActivateSheet(activeSheet.GetName)

Else

    'Discard the changes and reload the original sheet format template
    Dim reloadResult As swReloadTemplateResult_e
    reloadResult = activeSheet.ReloadTemplate(False)
    Debug.Print("Done - " & GetReloadResult(reloadResult))

End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Modify Break Corner Feature Example (C#)

This example shows how to create and modify a break corner feature in a sheet metal part.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified document.
// 2. Selects a face on Edge-Flange1.
// 3. Creates a break corner feature.
// 4. Unsuppresses the flat pattern feature.
// 5. Accesses the break corner feature and
//    and modifies it.
// 6. Suppresses the flat pattern feature.
// 7. Examine the graphics area and the Immediate window.
//
// NOTE: Because the part document is used elsewhere,
// do not save any changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace BreakCornerFeatureCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            BreakCornerFeatureData swBreakCornerFeatureData = default(BreakCornerFeatureData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\sm1-remove-edges.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", -0.111589911985732, 0.0979999999999563, 0.0841212722518208, true, 0, null, 0);
            swModel.InsertSheetMetalBreakCorner((int)swBreakCornerTypes_e.swBreakCornerTypeChamfer, 0.005);

            //Select and unsuppress the flat pattern feature
            status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swModel.EditUnsuppress2();
            swModel.ClearSelection2(true);

            //Select the break corner feature
            //and change some of its properties
            status = swModelDocExt.SelectByID2("Break-Corner1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, 0);
            swBreakCornerFeatureData = (BreakCornerFeatureData)swFeature.GetDefinition();
            status = swBreakCornerFeatureData.AccessSelections(swModel, null);
            Debug.Print("AccessSelections: " + status);
            Debug.Print("");
            Debug.Print(" -----Original-----");
            Debug.Print("    CenteredOnBendLines: " + swBreakCornerFeatureData.CenteredOnBendLines);
            Debug.Print("    InternalCornersOnly: " + swBreakCornerFeatureData.InternalCornersOnly);
            swBreakCornerFeatureData.InternalCornersOnly = true;
            swBreakCornerFeatureData.CenteredOnBendLines = true;
            Debug.Print("");
            Debug.Print(" -----Modified-----");
            Debug.Print("    CenteredOnBendLines: " + swBreakCornerFeatureData.CenteredOnBendLines);
            Debug.Print("    InternalCornersOnly: " + swBreakCornerFeatureData.InternalCornersOnly);
            status = swFeature.ModifyDefinition(swBreakCornerFeatureData, swModel, null);
            Debug.Print("");
            Debug.Print("ModifyDefinition: " + status);
            swModel.ClearSelection2(true);

            //Select and suppress the flat pattern feature
            status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swModel.EditSuppress2();
            swModel.ClearSelection2(true);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Modify Break Corner Feature Example (VBA)

This example shows how to create and modify a break corner feature in a sheet metal part.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified document.
' 2. Selects a face on Edge-Flange1.
' 3. Creates a break corner feature.
' 4. Unsuppresses the flat pattern feature.
' 5. Accesses the break corner feature and
'    and modifies it.
' 6. Suppresses the flat pattern feature.
' 7. Examine the graphics area and the Immediate window.
'
' NOTE: Because the part document is used elsewhere,
' do not save any changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swBreakCornerFeatureData As SldWorks.BreakCornerFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\sm1-remove-edges.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", -0.111589911985732, 9.799999999999563E-02, 8.41212722518208E-02, True, 0, Nothing, 0)
    swModel.InsertSheetMetalBreakCorner swBreakCornerTypes_e.swBreakCornerTypeChamfer, 0.005

    'Select and unsuppress the flat pattern feature
    status = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditUnsuppress2
    swModel.ClearSelection2 True

    'Select the break corner feature
    'and change some of its properties
    status = swModelDocExt.SelectByID("Break-Corner1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, 0)
    Set swBreakCornerFeatureData = swFeature.GetDefinition
    status = swBreakCornerFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "AccessSelections:", status
    Debug.Print ""
    Debug.Print " -----Original-----"
    Debug.Print "    CenteredOnBendLines:", swBreakCornerFeatureData.CenteredOnBendLines
    Debug.Print "    InternalCornersOnly:", swBreakCornerFeatureData.InternalCornersOnly
    swBreakCornerFeatureData.InternalCornersOnly = True
    swBreakCornerFeatureData.CenteredOnBendLines = True
    Debug.Print ""
    Debug.Print " -----Modified-----"
    Debug.Print "    CenteredOnBendLines:", swBreakCornerFeatureData.CenteredOnBendLines
    Debug.Print "    InternalCornersOnly:", swBreakCornerFeatureData.InternalCornersOnly
    status = swFeature.ModifyDefinition(swBreakCornerFeatureData, swModel, Nothing)
    Debug.Print ""
    Debug.Print "ModifyDefinition:", status
    swModel.ClearSelection2 True

    'Select and suppress the flat pattern feature
    status = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditSuppress2
    swModel.ClearSelection2 True

```

End Sub

Modify Break Corner Feature Example (VB.NET)

This example shows how to create and modify a break corner feature in a sheet metal part.

```

-----
Preconditions:
1. Verify that the specified part document exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified document.
2. Selects a face on Edge-Flange1.
3. Creates a break corner feature.
4. Unsuppresses the flat pattern feature.
5. Accesses the break corner feature and
   and modifies it.
6. Suppresses the flat pattern feature.
7. Examine the graphics area and the Immediate window.

NOTE: Because the part document is used elsewhere,
do not save any changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swBreakCornerFeatureData As BreakCornerFeatureData
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\sm1-remove-edges.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("", "FACE", -0.111589911985732, 0.09799999999999563, 0.0841212722518208, True, 0, Nothing, 0)
        swModel.InsertSheetMetalBreakCorner(swBreakCornerTypes_e.swBreakCornerTypeChamfer, 0.005)

        'Select and unsuppress the flat pattern feature
        status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditUnsuppress2()
        swModel.ClearSelection2(True)

        'Select the break corner feature
        'and change some of its properties
        status = swModelDocExt.SelectByID2("Break-Corner1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, 0)
        swBreakCornerFeatureData = swFeature.GetDefinition
        status = swBreakCornerFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("AccessSelections: " & status)
        Debug.Print("")
        Debug.Print("-----Original-----")
        Debug.Print("    CenteredOnBendLines: " & swBreakCornerFeatureData.CenteredOnBendLines)
        Debug.Print("    InternalCornersOnly: " & swBreakCornerFeatureData.InternalCornersOnly)
        swBreakCornerFeatureData.InternalCornersOnly = True
        swBreakCornerFeatureData.CenteredOnBendLines = True
        Debug.Print("")
        Debug.Print("-----Modified-----")
        Debug.Print("    CenteredOnBendLines: " & swBreakCornerFeatureData.CenteredOnBendLines)
        Debug.Print("    InternalCornersOnly: " & swBreakCornerFeatureData.InternalCornersOnly)
        status = swFeature.ModifyDefinition(swBreakCornerFeatureData, swModel, Nothing)
        Debug.Print("")
        Debug.Print("ModifyDefinition: " & status)
        swModel.ClearSelection2(True)

        'Select and suppress the flat pattern feature
        status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSuppress2()
        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Modify Chain Pattern Feature Example (C#)

This example shows how to modify a chain pattern feature.

```
//-----
// Preconditions:
// 1. Verify that these files exist in public_documents\samples\tutorial\api:
// * cam_roller.sldprt
// * distance_linkage.sldasm
// * mount_base.sldasm
// * mount_plate.sldprt
// 2. Open distance_linkage.sldasm.
// a. Click Insert > Component Pattern > Chain Pattern.
// 1. Click Distance Linkage in Pitch Method.
// 2. Click Belt Drive Path in the FeatureManager design tree
// for Path.
// 3. Set Number of Instances to 4.
// 4. Click Mount Base<1> in the FeatureManager design tree
// for Component to Pattern.
// 5. Set Spacing to approximately 10.
// 6. Click the cylindrical face on Cam Roller<3> (left-front cam)
// in the graphics area for Path Link1. Zoom in if necessary.
// 7. Click the cylindrical face on Cam Roller<1> (right-front cam)
// in the graphics area for Path Link2.
// 8. Expand Mount Base<1> and click its Right Plane in the
// FeatureManager design tree for Path Alignment Plane.
// b. Click OK to create ChainPattern1 feature.
// c. Click Zoom to Fit.
// 3. Examine the FeatureManager design tree and graphics area.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Selects the ChainPattern1 feature.
// 2. Gets and sets some ChainPattern1 data.
// 3. Click Zoom to fit.
// 4. Examine the Immediate window and graphics area.
//
// NOTE: Because these models are used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            ChainPatternFeatureData swChainPatternFeatureData = default(ChainPatternFeatureData);
            ModelView swModelView = default(ModelView);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelView = (ModelView)swModel.ActiveView;

            //Get ChainPattern1 feature
            //Get and set some of its data
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("ChainPattern1", "COMPATTERN", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swChainPatternFeatureData = (ChainPatternFeatureData)swFeature.GetDefinition();
            status = swChainPatternFeatureData.AccessSelections(swModel, null);
            Debug.Print("Name of feature: " + swFeature.Name);
            //Get pitch type
            Debug.Print(" Pitch type: " + swChainPatternFeatureData.PitchMethod);
            //Get align method
            Debug.Print(" Align method: " + swChainPatternFeatureData.AlignMethod);
            //Get whether Fill Path selected
            Debug.Print(" Original Fill Path: " + swChainPatternFeatureData.FillPath);
            //Get number of pattern instances and spacing
            Debug.Print(" Original number of instances: " + swChainPatternFeatureData.InstanceCount);
            Debug.Print(" Original distance between each pattern instance: " + swChainPatternFeatureData.Spacing);
            //Set Fill Path to true
            swChainPatternFeatureData.FillPath = true;
            Debug.Print(" Modified Fill Path: " + swChainPatternFeatureData.FillPath);
            //Change spacing
            swChainPatternFeatureData.Spacing = 0.2;
            Debug.Print(" Modified distance between each pattern instance: " + swChainPatternFeatureData.Spacing);

            swFeature.ModifyDefinition(swChainPatternFeatureData, swModel, null);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Modify Chain Pattern Feature Example (VBA)

This example shows how to modify a chain pattern feature.

```
'-----
' Preconditions:
' 1. Verify that these files exist in public_documents\samples\tutorial\api:
'    * cam roller.sldprt
'    * distance linkage.sldasm
'    * mount base.sldasm
'    * mount plate.sldprt
' 2. Open distance linkage.sldasm.
'    a. Click Insert > Component Pattern > Chain Pattern.
'       1. Click Distance Linkage in Pitch Method.
'       2. Click Belt Drive Path in the FeatureManager design tree
'          for Path.
'       3. Set Number of Instances to 4.
'       4. Click Mount Base<1> in the FeatureManager design tree
'          for Component to Pattern.
'       5. Set Spacing to approximately 10.
'       6. Click the cylindrical face on Cam Roller<3> (left-front cam)
'          in the graphics area for Path Link1. Zoom in if necessary.
'       7. Click the cylindrical face on Cam Roller<1> (right-front cam)
'          in the graphics area for Path Link2.
'       8. Expand Mount Base<1> and click its Right Plane in the
'          FeatureManager design tree for Path Alignment Plane.
'    b. Click OK to create ChainPattern1 feature.
'    c. Click Zoom to Fit.
' 3. Examine the FeatureManager design tree and graphics area.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Selects the ChainPattern1 feature.
' 2. Gets and sets some ChainPattern1 data.
' 3. Click Zoom to fit.
' 4. Examine the Immediate window and graphics area.
'
' NOTE: Because these models are used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swChainPatternFeatureData As SldWorks.ChainPatternFeatureData
Dim swModelView As SldWorks.ModelView
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelView = swModel.ActiveView

    'Get ChainPattern1 feature
    'Get and set some of its data
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("ChainPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swChainPatternFeatureData = swFeature.GetDefinition
    status = swChainPatternFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "Name of feature: " & swFeature.Name
    'Get pitch type
    Debug.Print " Pitch type: " & swChainPatternFeatureData.PitchMethod
    'Get align method
    Debug.Print " Align method: " & swChainPatternFeatureData.AlignMethod
    'Get whether Fill Path selected
    Debug.Print " Original Fill Path: " & swChainPatternFeatureData.FillPath
    'Get number of pattern instances and spacing
    Debug.Print " Original number of instances: " & swChainPatternFeatureData.InstanceCount
    Debug.Print " Original distance between each pattern instance: " & swChainPatternFeatureData.Spacing
    'Set Fill Path to true
    swChainPatternFeatureData.FillPath = True
    Debug.Print " Modified Fill Path: " & swChainPatternFeatureData.FillPath
    'Change spacing
    swChainPatternFeatureData.Spacing = 0.2
    Debug.Print " Modified distance between each pattern instance: " & swChainPatternFeatureData.Spacing

    swFeature.ModifyDefinition swChainPatternFeatureData, swModel, Nothing

End Sub
```

Modify Chain Pattern Feature Example (VB.NET)

This example shows how to modify a chain pattern feature.

```

-----
Preconditions:
1. Verify that these files exist in public_documents\samples\tutorial\api:
   * cam_roller.sldprt
   * distance_linkage.sldasm
   * mount_base.sldasm
   * mount_plate.sldprt
2. Open distance_linkage.sldasm.
   a. Click Insert > Component Pattern > Chain Pattern.
      1. Click Distance Linkage in Pitch Method.
      2. Click Belt Drive Path in the FeatureManager design tree for Path.
      3. Set Number of Instances to 4.
      4. Click Mount Base<1> in the FeatureManager design tree for Component to Pattern.
      5. Set Spacing to approximately 10.
      6. Click the cylindrical face on Cam Roller<3> (left-front cam) in the graphics area for Path Link1. Zoom in if necessary.
      7. Click the cylindrical face on Cam Roller<1> (right-front cam) in the graphics area for Path Link2.
      8. Expand Mount Base<1> and click its Right Plane in the FeatureManager design tree for Path Alignment Plane.
   b. Click OK to create ChainPattern1 feature.
   c. Click Zoom to Fit.
3. Examine the FeatureManager design tree and graphics area.
4. Open the Immediate window.

Postconditions:
1. Selects the ChainPattern1 feature.
2. Gets and sets some ChainPattern1 data.
3. Click Zoom to fit.
4. Examine the Immediate window and graphics area.

-----
NOTE: Because these models are used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swChainPatternFeatureData As ChainPatternFeatureData
        Dim swModelView As ModelView
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swModelView = swModel.ActiveView

        'Get ChainPattern1 feature
        'Get and set some of its data
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("ChainPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swChainPatternFeatureData = swFeature.GetDefinition
        status = swChainPatternFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Name of feature: " & swFeature.Name)
        'Get pitch type
        Debug.Print(" Pitch type: " & swChainPatternFeatureData.PitchMethod)
        'Get align method
        Debug.Print(" Align method: " & swChainPatternFeatureData.AlignMethod)
        'Get whether Fill Path selected
        Debug.Print(" Original Fill Path: " & swChainPatternFeatureData.FillPath)
        'Get number of pattern instances and spacing
        Debug.Print(" Original number of instances: " & swChainPatternFeatureData.InstanceCount)
        Debug.Print(" Original distance between each pattern instance: " & swChainPatternFeatureData.Spacing)
        'Set Fill Path to true
        swChainPatternFeatureData.FillPath = True
        Debug.Print(" Modified Fill Path: " & swChainPatternFeatureData.FillPath)
        'Change spacing
        swChainPatternFeatureData.Spacing = 0.2
        Debug.Print(" Modified distance between each pattern instance: " & swChainPatternFeatureData.Spacing)

        swFeature.ModifyDefinition(swChainPatternFeatureData, swModel, Nothing)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Modify Derived Part Example (C#)

This example shows how to insert and modify a derived part.

```
//-----
// Preconditions:
// 1. Verify that the part documents to open and insert exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Inserts a derived part in the part document
//    opened in step 1.
// 3. Changes some parameters of the derived part feature.
// 4. Examine the Immediate window.
//
// NOTE: Because both part documents are used elsewhere,
// do not save any changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DerivedPartFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Component2 swComp = default(Component2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swDerivedFeat = default(Feature);
            Feature swFeat = default(Feature);
            DerivedPartFeatureData swDerivedData = default(DerivedPartFeatureData);
            bool bRet = false;
            string fileName = null;
            string derivedFileName = null;
            int errors = 0;
            int warnings = 0;

            //Open part, insert another part, and select the derived part feature
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\holecube.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            derivedFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.sldprt";
            swPart = (PartDoc)swModel;
            swFeat = (Feature)swPart.InsertPart3(derivedFileName, (int)swInsertPartOptions_e.swInsertPartImportSolids, "Default" );
            swModel.ClearSelection2(true);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SelectByID2("box", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            //Get the selected derived part feature
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swDerivedFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent3(1, -1);
            Debug.Print("Name of derived part feature = " + swDerivedFeat.Name);
            Debug.Print("");
            //Get and modify import planes with derived part
            bRet = TestImportPlane(swDerivedFeat, swModel, swComp);
            Debug.Print("");
            //Get and modify import absorbed sketches with derived part
            bRet = TestImportAbsorbedSketches(swDerivedFeat, swModel, swComp);
            Debug.Print("");
            //Get and modify import unabsorbed sketches with derived part
            bRet = TestImportUnAbsorbedSketches(swDerivedFeat, swModel, swComp);
            Debug.Print("");
            swDerivedData = (DerivedPartFeatureData)swDerivedFeat.GetDefinition();
            Debug.Print("Derived file's path and name = " + swDerivedData.PathName);
        }

        public bool TestImportPlane(Feature feat, ModelDoc2 doc, Component2 comp)
        {
            DerivedPartFeatureData featData = default(DerivedPartFeatureData);
            bool startVal = false;
            bool boolstatus = false;

            featData = (DerivedPartFeatureData)feat.GetDefinition();
            startVal = featData.ImportPlane;
            Debug.Print("Import planes with derived part? " + startVal);
            featData.ImportPlane = true;
            Debug.Print("Modified import planes with derived part? " + featData.ImportPlane);
            boolstatus = feat.ModifyDefinition(featData, doc, comp);
            featData = null;
            return boolstatus;
        }

        public bool TestImportAbsorbedSketches(Feature feat, ModelDoc2 doc, Component2 comp)
        {
            DerivedPartFeatureData featData = default(DerivedPartFeatureData);
            bool startVal = false;
            bool boolstatus = false;

            featData = (DerivedPartFeatureData)feat.GetDefinition();
            startVal = featData.ImportAbsorbedSketches;
            Debug.Print("Import absorbed sketches with derived part? " + startVal);
            featData.ImportAbsorbedSketches = true;
            Debug.Print("Modified import absorbed sketches with derived part? " + featData.ImportAbsorbedSketches);
            boolstatus = feat.ModifyDefinition(featData, doc, comp);
            featData = null;
            return boolstatus;
        }

        public bool TestImportUnAbsorbedSketches(Feature feat, ModelDoc2 doc, Component2 comp)
```

```

{
    DerivedPartFeatureData featData = default(DerivedPartFeatureData);
    bool startVal = false;
    bool boolstatus = false;

    featData = (DerivedPartFeatureData)feat.GetDefinition();
    startVal = featData.ImportUnAbsorbedSketches;
    Debug.Print("Import unabsorbed sketches with derived part? " + startVal);
    featData.ImportUnAbsorbedSketches = true;
    Debug.Print("Modified import unabsorbed sketches with derived part? " + featData.ImportUnAbsorbedSketches);
    boolstatus = feat.ModifyDefinition(featData, doc, comp);
    featData = null;
    return boolstatus;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Modify Derived Part Example (VBA)

This example shows how to insert and modify a derived part.

```
'-----
' Preconditions:
' 1. Verify that the part documents to open and insert exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Inserts a derived part in the part document
'    opened in step 1.
' 3. Changes some parameters of the derived part feature.
' 4. Examine the Immediate window.
'
' NOTE: Because both part documents are used elsewhere,
' do not save any changes.
'-----
```

Option Explicit

```
Function TestImportPlane(feas As Feature, doc As ModelDoc2, comp As Component2) As Boolean
    Dim featData As SldWorks.DerivedPartFeatureData
    Dim startVal As Boolean
    Dim boolstatus As Boolean
```

```
    Set featData = feas.GetDefinition
    startVal = featData.ImportPlane
    Debug.Print "Import planes with derived part? " & startVal
    featData.ImportPlane = True
    Debug.Print "Modified import planes with derived part? " & featData.ImportPlane
    boolstatus = feat.ModifyDefinition(featData, doc, comp)
    Set featData = Nothing
End Function
```

```
Function TestImportAbsorbedSketches(feas As Feature, doc As ModelDoc2, comp As Component2) As Boolean
    Dim featData As SldWorks.DerivedPartFeatureData
    Dim startVal As Boolean
    Dim boolstatus As Boolean
```

```
    Set featData = feas.GetDefinition
    startVal = featData.ImportAbsorbedSketches
    Debug.Print "Import absorbed sketches with derived part? " & startVal
    featData.ImportAbsorbedSketches = True
    Debug.Print "Modified import absorbed sketches with derived part? " & featData.ImportAbsorbedSketches
    boolstatus = feat.ModifyDefinition(featData, doc, comp)
    Set featData = Nothing
End Function
```

```
Function TestImportUnAbsorbedSketches(feas As Feature, doc As ModelDoc2, comp As Component2) As Boolean
    Dim featData As SldWorks.DerivedPartFeatureData
    Dim startVal As Boolean
    Dim boolstatus As Boolean
```

```
    Set featData = feas.GetDefinition
    startVal = featData.ImportUnAbsorbedSketches
    Debug.Print "Import unabsorbed sketches with derived part? " & startVal
    featData.ImportUnAbsorbedSketches = True
    Debug.Print "Modified import unabsorbed sketches with derived part? " & featData.ImportUnAbsorbedSketches
    boolstatus = feat.ModifyDefinition(featData, doc, comp)
    Set featData = Nothing
End Function
```

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swComp As SldWorks.Component2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDerivedFeat As SldWorks.Feature
    Dim swFeat As SldWorks.Feature
    Dim swDerivedData As SldWorks.DerivedPartFeatureData
    Dim bRet As Boolean
    Dim fileName As String
    Dim derivedFileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = Application.SldWorks

    'Open part, insert another part, and select the derived part feature
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\holecube.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    derivedFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
    Set swPart = swModel
    Set swFeat = swPart.InsertPart3(derivedFileName, swInsertPartOptions_e.swInsertPartImportSolids, "Default")
    swModel.ClearSelection2 True
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SelectByID2("box", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

    'Get the selected derived part feature
    Set swSelMgr = swModel.SelectionManager
    Set swDerivedFeat = swSelMgr.GetSelectedObject6(1, -1)
```

```
Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1)
Debug.Print "Name of derived part feature = " & swDerivedFeat.Name

Debug.Print ""
'Get and modify import planes with derived part
bRet = TestImportPlane(swDerivedFeat, swModel, swComp)
Debug.Print ""
'Get and modify import absorbed sketches with derived part
bRet = TestImportAbsorbedSketches(swDerivedFeat, swModel, swComp)
Debug.Print ""
'Get and modify import unabsorbed sketches with derived part
bRet = TestImportUnAbsorbedSketches(swDerivedFeat, swModel, swComp)

Debug.Print ""
Set swDerivedData = swDerivedFeat.GetDefinition
Debug.Print "Derived file's path and name = " & swDerivedData.PathName
```

End Sub

Modify Derived Part Example (VB.NET)

This example shows how to insert and modify a derived part.

```

-----
Preconditions:
1. Verify that the part documents to open and insert exist.
2. Open the Immediate window.
Postconditions:
1. Opens the specified part document.
2. Inserts a derived part in the part document
   opened in step 1.
3. Changes some parameters of the derived part feature.
4. Examine the Immediate window.
NOTE: Because both part documents are used elsewhere,
do not save any changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swComp As Component2
        Dim swSelMgr As SelectionMgr
        Dim swDerivedFeat As Feature
        Dim swFeat As Feature
        Dim swDerivedData As DerivedPartFeatureData
        Dim bRet As Boolean
        Dim fileName As String
        Dim derivedFileName As String
        Dim errors As Integer
        Dim warnings As Integer

        'Open part, insert another part, and select the derived part feature
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\holecube.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        derivedFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
        swPart = swModel
        swFeat = swPart.InsertPart3(derivedFileName, swInsertPartOptions_e.swInsertPartImportSolids, "Default")
        swModel.ClearSelection2(True)
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID2("box", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        'Get the selected derived part feature
        swSelMgr = swModel.SelectionManager
        swDerivedFeat = swSelMgr.GetSelectedObject6(1, -1)
        swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1)
        Debug.Print("Name of derived part feature = " & swDerivedFeat.Name)
        Debug.Print("")
        'Get and modify import planes with derived part
        bRet = TestImportPlane(swDerivedFeat, swModel, swComp)
        Debug.Print("")
        'Get and modify import absorbed sketches with derived part
        bRet = TestImportAbsorbedSketches(swDerivedFeat, swModel, swComp)
        Debug.Print("")
        'Get and modify import unabsorbed sketches with derived part
        bRet = TestImportUnAbsorbedSketches(swDerivedFeat, swModel, swComp)
        Debug.Print("")
        swDerivedData = swDerivedFeat.GetDefinition
        Debug.Print("Derived file's path and name = " & swDerivedData.PathName)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

    Function TestImportPlane(ByVal feat As Feature, ByVal doc As ModelDoc2, ByVal comp As Component2) As Boolean
        Dim featData As DerivedPartFeatureData
        Dim startVal As Boolean
        Dim boolstatus As Boolean

        featData = feat.GetDefinition
        startVal = featData.ImportPlane
        Debug.Print("Import planes with derived part? " & startVal)
        featData.ImportPlane = True
        Debug.Print("Modified import planes with derived part? " & featData.ImportPlane)
        boolstatus = feat.ModifyDefinition(featData, doc, comp)
        featData = Nothing
    End Function

    Function TestImportAbsorbedSketches(ByVal feat As Feature, ByVal doc As ModelDoc2, ByVal comp As Component2) As Boolean
        Dim featData As DerivedPartFeatureData
        Dim startVal As Boolean
        Dim boolstatus As Boolean

        featData = feat.GetDefinition
        startVal = featData.ImportAbsorbedSketches
        Debug.Print("Import absorbed sketches with derived part? " & startVal)
        featData.ImportAbsorbedSketches = True
        Debug.Print("Modified import absorbed sketches with derived part? " & featData.ImportAbsorbedSketches)
        boolstatus = feat.ModifyDefinition(featData, doc, comp)
        featData = Nothing
    End Function

    Function TestImportUnAbsorbedSketches(ByVal feat As Feature, ByVal doc As ModelDoc2, ByVal comp As Component2) As Boolean
        Dim featData As DerivedPartFeatureData
        Dim startVal As Boolean
        Dim boolstatus As Boolean
    
```

```
    featData = feat.GetDefinition
    startVal = featData.ImportUnAbsorbedSketches
    Debug.Print("Import unabsorbed sketches with derived part? " & startVal)
    featData.ImportUnAbsorbedSketches = True
    Debug.Print("Modified import unabsorbed sketches with derived part? " & featData.ImportUnAbsorbedSketches)
    boolstatus = feat.ModifyDefinition(featData, doc, comp)
    featData = Nothing
End Function

End Class
```

Modify Dimension Properties Example (C#)

This example shows how to modify the properties of a selected display dimension in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing document with one or more display
//    dimensions.
// 2. Select a display dimension.
//
// Postconditions:
// 1. The properties of the selected display dimension
//    are modified as specified.
// 2. Examine the display dimension to verify.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace ModifyDimensionProperties_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            boolstatus = swModelDocExt.EditDimensionProperties((int)swTolType_e.swTolBASIC, 0, 0, "", "", true, 9,
(int)swDimensionArrowsSide_e.swDimArrowsSmart, false, (int)swArrowStyle_e.swSLASH_ARROWHEAD, (int)swArrowStyle_e.swSLASH_ARROWHEAD,
"", "", true, "", "", "Example of lower text", true, (int)swInConfigurationOpts_e.swThisConfiguration, "");

            swModel.ClearSelection2(true);

        }

        public SldWorks swApp;
    }
}
```

Modify Dimension Properties Example (VBA)

This example shows how to modify the properties of a selected display dimension in a drawing.

```
' -----  
' Preconditions:  
' 1. Open a drawing document with one or more display  
'    dimensions.  
' 2. Select a display dimension.  
'  
' Postconditions:  
' 1. The properties of the selected display dimension  
'    are modified as specified.  
' 2. Examine the display dimension to verify.  
' -----  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim boolstatus As Boolean  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    boolstatus = swModelDocExt.EditDimensionProperties(swTolBASIC, 0, 0, "", "", True, 9, swDimArrowsSmart, False, swSLASH_ARROWHEAD, swSLASH_ARROWHEAD,  
    "", "", True, "", "", "Example of lower text", True, swThisConfiguration, "")  
  
    swModel.ClearSelection2 True  
  
End Sub
```

Modify Dimension Properties Example (VB.NET)

This example shows how to modify the properties of a selected display dimension in a drawing.

```
'-----  
' Preconditions:  
' 1. Open a drawing document with one or more display  
'    dimensions.  
' 2. Select a display dimension.  
'  
' Postconditions:  
' 1. The properties of the selected display dimension  
'    are modified as specified.  
' 2. Examine the display dimension to verify.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2  
    Dim swModelDoc Ext As ModelDocExtension  
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc  
        swModelDocExt = swModel.Extension
```

```
        boolstatus = swModelDocExt.EditDimensionProperties(swTolType_e.swTolBASIC, 0, 0, "", "", True, 9,  
swDimensionArrowsSide_e.swDimArrowsSmart, False, swArrowStyle_e.swSLASH_ARROWHEAD, swArrowStyle_e.swSLASH_ARROWHEAD, "", "", True, "",  
"", "Example of lower text", True, swInConfigurationOpts_e.swThisConfiguration, ""  
)
```

```
        swModel.ClearSelection2(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Modify Fillet Weld Bead Example (C#)

This example shows how to modify a fillet weld bead..

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Inspect the Immediate window to see the properties of Fillet Bead1.
// 2. Modifies some properties of Fillet Bead1.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetWeldBeadFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelMgr;
        WeldmentBeadFeatureData swWeldBead;
        Feature swFeat;
        SelectData swSelData;
        Vertex v1;
        Vertex v2;
        object[] set1;
        object[] faceVar;
        object[] ve;
        object fVar1;
        object fVar2;
        object[] f1 = new object[1];
        object[] f2 = new object[2];
        object[] e = new object[1];
        double bdlen;
        double bdPitch;
        double bdsz;
        int bdTy;
        int i;
        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = swSelMgr.CreateSelectData();

            //Select Fillet Bead1 feature
            boolstatus = swModelDocExt.SelectByID("Fillet Bead1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swWeldBead = (WeldmentBeadFeatureData)swFeat.GetDefinition();

            //Roll back to the feature just before Fillet Bead1
            boolstatus = swWeldBead.AccessSelections(swModel, null);

            //Get Fillet Bead1 properties
            Debug.Print("Fillet Bead1 properties:");
            bdlen = swWeldBead.get_BeadLength((int)swWeldBeadSide_e.swWeldBeadArrowSide);
            Debug.Print(" Weld bead length: " + bdlen);

            bdPitch = swWeldBead.get_BeadPitch((int)swWeldBeadSide_e.swWeldBeadArrowSide);
            Debug.Print(" Weld bead pitch: " + bdPitch);

            bdsz = swWeldBead.get_BeadSize((int)swWeldBeadSide_e.swWeldBeadArrowSide);
            Debug.Print(" Weld bead size: " + bdsz);

            bdTy = swWeldBead.get_BeadType((int)swWeldBeadSide_e.swWeldBeadArrowSide);
            Debug.Print(" Weld bead type as defined in swWeldBeadType_e: " + bdTy);

            Debug.Print(" Propagate the weld bead along the tangent? " + swWeldBead.TangentPropagation);
            Debug.Print(" Apply weld bead to both sides of intersecting faces? " + swWeldBead.UseOtherSide);

            //Get Fillet Bead1 faces
            swWeldBead.GetFaces((int)swWeldBeadSide_e.swWeldBeadArrowSide, out fVar1, out fVar2);
            faceVar = (object[])fVar2;
            set1 = (object[])fVar1;
        }
    }
}
```

```

    for (i = faceVar.GetLowerBound(0); i <= faceVar.GetUpperBound(0); i++)
    {
        ((Entity)faceVar[i]).Select4(true, swSelData);
    }

    for (i = set1.GetLowerBound(0); i <= set1.GetUpperBound(0); i++)
    {
        ((Entity)set1[i]).Select4(true, swSelData);
    }

    //Get Fillet Bead1 virtual edges
    ve = (object[])swWeldBead.GetVirtualEdges(false, (int)swWeldBeadSide_e.swWeldBeadArrowSide);

    for (i = ve.GetLowerBound(0); i <= ve.GetUpperBound(0); i++)
    {
        boolstatus = ((Entity)ve[i]).Select4(true, swSelData);
        v1 = (Vertex)((Edge)ve[i]).GetStartVertex();
        v2 = (Vertex)((Edge)ve[i]).GetEndVertex();
    }

    swModel.ClearSelection2(true);

    //Set new properties of Fillet Bead1
    swWeldBead.set_BeadLength((int)swWeldBeadSide_e.swWeldBeadArrowSide, bdlen * 0.5);
    swWeldBead.set_BeadPitch((int)swWeldBeadSide_e.swWeldBeadArrowSide, bdPitch * 0.5);
    swWeldBead.set_BeadSize((int)swWeldBeadSide_e.swWeldBeadArrowSide, bdsz * 0.5);
    swWeldBead.set_BeadType((int)swWeldBeadSide_e.swWeldBeadArrowSide, bdTy);

    //Modify Fillet Bead1
    boolstatus = swFeat.ModifyDefinition(swWeldBead, swModel, null);

}

public SldWorks swApp;

}

```

Modify Fillet Weld Bead Example (VBA)

This example shows how to modify a fillet weld bead.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window to see the properties of Fillet Bead1.
' 2. Modifies some properties of Fillet Bead1.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swWeldBead As SldWorks.WeldmentBeadFeatureData
Dim swFeat As SldWorks.Feature
Dim swSelData As SldWorks.SelectData
Dim v1 As SldWorks.Vertex
Dim v2 As SldWorks.Vertex
Dim set1 As Variant
Dim faceVar As Variant
Dim ve As Variant
Dim fVar1 As Variant
Dim fVar2 As Variant
Dim f1(0) As Object
Dim f2(1) As Object
Dim e(0) As Object
Dim bdlen As Double
Dim bdPitch As Double
Dim bdsz As Double
Dim bdTy As Long
Dim i As Long
Dim boolstatus As Boolean
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager
Set swSelData = swSelMgr.CreateSelectData
```

```
'Select Fillet Bead1 feature
boolstatus = swModelDocExt.SelectByID2("Fillet Bead1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swWeldBead = swFeat.GetDefinition
```

```
'Roll back to the feature just before Fillet Bead1
boolstatus = swWeldBead.AccessSelections(swModel, Nothing)
```

```
'Get Fillet Bead1 properties
Debug.Print "Fillet Bead1 properties:"
bdlen = swWeldBead.BeadLength(swWeldBead.ArrowSide)
Debug.Print " Weld bead length: " & bdlen
```

```
bdPitch = swWeldBead.BeadPitch(swWeldBead.ArrowSide)
Debug.Print " Weld bead pitch: " & bdPitch
```

```
bdsz = swWeldBead.BeadSize(swWeldBead.ArrowSide)
Debug.Print " Weld bead size: " & bdsz
```

```
bdTy = swWeldBead.BeadType(swWeldBead.ArrowSide)
Debug.Print " Weld bead type as defined in swWeldBeadType_e: " & bdTy
```

```
Debug.Print " Propagate the weld bead along the tangent? " & swWeldBead.TangentPropagation
Debug.Print " Apply weld bead to both sides of intersecting faces? " & swWeldBead.UseOtherSide
```

```
'Get Fillet Bead1 faces
swWeldBead.GetFaces swWeldBead.ArrowSide, set1, faceVar
```

```
For i = LBound(faceVar) To UBound(faceVar)
faceVar(i).Select4 True, swSelData
Next i
```

```
For i = LBound(set1) To UBound(set1)
set1(i).Select4 True, swSelData
```

Next i

```
'Get Fillet Bead1 virtual edges  
ve = swWeldBead.GetVirtualEdges(False, swWeldBeadArrowSide)
```

```
For i = LBound(ve) To UBound(ve)  
    boolstatus = ve(i).Select4(True, swSelData)  
    Set v1 = ve(i).GetStartVertex  
    Set v2 = ve(i).GetEndVertex  
Next i
```

```
swModel.ClearSelection2 True
```

```
'Set new properties of Fillet Bead1  
swWeldBead.BeadLength(swWeldBeadArrowSide) = bdlen * 0.5  
swWeldBead.BeadPitch(swWeldBeadArrowSide) = bdPitch * 0.5  
swWeldBead.BeadSize(swWeldBeadArrowSide) = bdsz * 0.5  
swWeldBead.BeadType(swWeldBeadArrowSide) = bdTy
```

```
'Modify Fillet Bead1  
boolstatus = swFeat.ModifyDefinition(swWeldBead, swModel, Nothing)
```

End Sub

Modify Fillet Weld Bead Example (VB.NET)

This example shows how to modify a fillet weld bead.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window to see the properties of Fillet Bead1.
' 2. Modifies some properties of Fillet Bead1.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim swWeldBead As WeldmentBeadFeatureData
    Dim swFeat As Feature
    Dim swSelData As SelectData
    Dim v1 As Vertex
    Dim v2 As Vertex
    Dim set1 As Object
    Dim faceVar As Object
    Dim ve As Object
    Dim fVar1 As Object
    Dim fVar2 As Object
    Dim f1(0) As Object
    Dim f2(1) As Object
    Dim e(0) As Object
    Dim bdlen As Double
    Dim bdPitch As Double
    Dim bdsz As Double
    Dim bdTy As Integer
    Dim i As Integer
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData

        'Select Fillet Bead1 feature
        boolstatus = swModelDocExt.SelectByID2("Fillet Bead1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swWeldBead = swFeat.GetDefinition

        'Roll back to the feature just before Fillet Bead1
        boolstatus = swWeldBead.AccessSelections(swModel, Nothing)

        'Get Fillet Bead1 properties
        Debug.Print("Fillet Bead1 properties:")
        bdlen = swWeldBead.BeadLength(swWeldBeadSide_e.swWeldBeadArrowSide)
        Debug.Print(" Weld bead length: " & bdlen)

        bdPitch = swWeldBead.BeadPitch(swWeldBeadSide_e.swWeldBeadArrowSide)
        Debug.Print(" Weld bead pitch: " & bdPitch)

        bdsz = swWeldBead.BeadSize(swWeldBeadSide_e.swWeldBeadArrowSide)
        Debug.Print(" Weld bead size: " & bdsz)

        bdTy = swWeldBead.BeadType(swWeldBeadSide_e.swWeldBeadArrowSide)
        Debug.Print(" Weld bead type as defined in swWeldBeadType_e: " & bdTy)

        Debug.Print(" Propagate the weld bead along the tangent? " & swWeldBead.TangentPropagation)
        Debug.Print(" Apply weld bead to both sides of intersecting faces? " & swWeldBead.UseOtherSide)

        'Get Fillet Bead1 faces
        swWeldBead.GetFaces(swWeldBeadSide_e.swWeldBeadArrowSide, set1, faceVar)

        For i = LBound(faceVar) To UBound(faceVar)
            faceVar(i).Select4(True, swSelData)
        Next i

        For i = LBound(set1) To UBound(set1)
            set1(i).Select4(True, swSelData)
        Next i
    End Sub

```

```

'Get Fillet Bead1 virtual edges
ve = swWeldBead.GetVirtualEdges(False, swWeldBeadSide_e.swWeldBeadArrowSide)

For i = LBound(ve) To UBound(ve)
    boolstatus = ve(i).Select4(True, swSelData)
    v1 = ve(i).GetStartVertex
    v2 = ve(i).GetEndVertex
Next i

swModel.ClearSelection2(True)

'Set new properties of Fillet Bead1
swWeldBead.BeadLength(swWeldBeadSide_e.swWeldBeadArrowSide) = bdlen * 0.5
swWeldBead.BeadPitch(swWeldBeadSide_e.swWeldBeadArrowSide) = bdPitch * 0.5
swWeldBead.BeadSize(swWeldBeadSide_e.swWeldBeadArrowSide) = bdsz * 0.5
swWeldBead.BeadType(swWeldBeadSide_e.swWeldBeadArrowSide) = bdTy

'Modify Fillet Bead1
boolstatus = swFeat.ModifyDefinition(swWeldBead, swModel, Nothing)

```

End Sub

Public swApp As SldWorks

End Class

Modify Heal Edges Feature Example (VBA)

This example shows how to modify an existing heal edges feature.

```
'-----
' Preconditions:
' 1. Open a part document that contains
'    an imported part with the edges of four faces
'    merged in a heal edges feature.
' 2. Edit the heal edges feature to verify that
'    four faces are listed in Faces in the Heal
'    Edges PropertyManager page.
' 3. Close the PropertyManager page.
'
' Postconditions:
' 1. At stop, press the Ctrl key while selecting
'    two faces, then press F5.
' 2. Modifies the heal edges feature
'    so that the edges of the two selected
'    faces are merged.
' 3. Edit the heal edges feature to verify
'    that only two faces are listed in Faces
'    in the Heal Edges PropertyManager page and
'    examine the graphics area.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swHealEdgesFeature As SldWorks.HealEdgesFeatureData
Dim swFeature As SldWorks.Feature
Dim pFaceArr(1) As Object
Dim vNewFaces As Variant
Dim boolstatus As Boolean
Dim nBefore As Long, nAfter As Long

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID2("HealEdge1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True
    Set swHealEdgesFeature = swFeature.GetDefinition

    swHealEdgesFeature.AccessSelections swModel, Nothing
    Stop
    'Select two faces (press the Ctrl key while selecting the faces)
    'Press F5

    Set pFaceArr(0) = swSelMgr.GetSelectedObject6(1, 0)
    Set pFaceArr(1) = swSelMgr.GetSelectedObject6(2, 0)
    swModel.ClearSelection2 True

    vNewFaces = pFaceArr
    swHealEdgesFeature.Faces = (vNewFaces)
    swHealEdgesFeature.HealEdges
    swHealEdgesFeature.GetEdgeInformation nBefore, nAfter
    swModel.ClearSelection2 True

    swFeature.ModifyDefinition swHealEdgesFeature, swModel, Nothing

End Sub
```

Modify Multiple Drawing Sheets Setups Example (C#)

This example shows how to modify the setups of multiple drawing sheets.

```
//-----
// Preconditions:
// 1. Verify that the drawing and sheet format files exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the drawing.
// 2. Sets Sheet1, Sheet2 and Sheet3 drawing sheet formats
//    to portrait.
// 3. Rebuilds the drawing.
// 4. Click each sheet tab, click Zoom to Fit, and examine
//    the sheet.
//
// NOTE: Because the drawing is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            object sheetNameArray = null;
            string[] sheetNames = new string[2];

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swDrawing = (DrawingDoc)swModel;
            sheetNames[0] = "Sheet2";
            sheetNames[1] = "Sheet3";
            sheetNameArray = sheetNames;
            swDrawing.SetSheetsSelected(sheetNameArray);
            status = swDrawing.SetupSheet6("Sheet3", (int)swDwgPaperSizes_e.swDwgPapersUserDefined, (int)swDwgTemplates_e.swDwgTemplateCustom, 1, 1, true, "C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2017\\lang\\english\\sheetformat\\a4 - portrait.slddrt", 0.2794, 0.2159, "Default", true, 0, 0, 0, 0, 0);

            swModel.ForceRebuild3(true);
            swModel.ViewZoomtofit2();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Modify Multiple Drawing Sheets Setups Example (VBA)

This example shows how to modify the setups of multiple drawing sheets.

```
'-----
' Preconditions:
' 1. Verify that the drawing and sheet format files exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Sets Sheet1, Sheet2 and Sheet3 drawing sheet formats
'    to portrait.
' 3. Rebuilds the drawing.
' 4. Click each sheet tab, click Zoom to Fit, and examine
'    the sheet.
'
' NOTE: Because the drawing is used elsewhere, do not
' save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim sheetNameArray As Variant
Dim sheetNames(1) As String
```

Sub main()

```
    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swDrawing = swModel

    sheetNames(0) = "Sheet2"
    sheetNames(1) = "Sheet3"
    sheetNameArray = sheetNames
    swDrawing.SetSheetsSelected (sheetNameArray)
    status = swDrawing.SetupSheet6("Sheet3", swDwgPaperSizes_e.swDwgPapersUserDefined, swDwgTemplates_e.swDwgTemplateCustom, 1, 1, True,
"C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\lang\english\sheetformat\A4 - portrait.slddrt", 0.2794, 0.2159, "Default", True, 0, 0, 0, 0, 0)

    swModel.ForceRebuild3 True
    swModel.ViewZoomtofit2
```

End Sub

Modify Multiple Drawing Sheets Setups Example (VB.NET)

This example shows how to modify the setups of multiple drawing sheets.

```

-----
Preconditions:
1. Verify that the drawing and sheet format files exist.
2. Open the Immediate window.

Postconditions:
1. Opens the drawing.
2. Sets Sheet1, Sheet2 and Sheet3 drawing sheet formats
   to portrait.
3. Rebuilds the drawing.
4. Click each sheet tab, click Zoom to Fit, and examine
   the sheet.

NOTE: Because the drawing is used elsewhere, do not
save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim sheetNameArray As Object
        Dim sheetNames(1) As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swDrawing = swModel
        sheetNames(0) = "Sheet2"
        sheetNames(1) = "Sheet3"
        sheetNameArray = sheetNames
        swDrawing.SetSheetsSelected(sheetNameArray)
        status = swDrawing.SetupSheet6("Sheet3", swDwgPaperSizes_e.swDwgPapersUserDefined, swDwgTemplates_e.swDwgTemplateCustom, 1, 1, True, "C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\lang\english\sheetformat\A4 - portrait.slddrt", 0.2794, 0.2159, "Default", True, 0, 0, 0, 0, 0)

        swModel.ForceRebuild3(True)
        swModel.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Modify Plane by Changing System Value Example (VBA)

This example shows how to modify the offset distance of an offset plane by modifying the system value.

```
'-----  
' Preconditions:  
' 1. Open a model document that has an offset  
'    plane named Plane1.  
' 2. Note the location of Plane1.  
'  
' Postconditions:  
' 1. Halves the offset distance of Plane1.  
' 2. Examine the graphics area and note the  
'    the location of Plane1.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swDim As SldWorks.Dimension  
Dim dimValue As Variant  
Dim boolstatus As Boolean  
Dim longstatus As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
    boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)  
    Set swDim = swModel.Parameter("D1@Plane1")  
    dimValue = swDim.GetSystemValue3(swThisConfiguration, Empty)  
    dimValue(0) = dimValue(0) / 2  
    longstatus = swDim.SetSystemValue3(dimValue(0), swSetValue_InThisConfiguration, Empty)  
  
    swModel.EditRebuild3  
  
End Sub
```

Modify Plane by Editing Its Definition Example (VBA)

This example shows how to modify a plane by editing its definition.

```
'-----  
' Preconditions:  
' 1. Open a model document with an offset plane  
'    named Plane1.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Changes the the offset distance of Plane1 to  
'    100mm.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swRefPlane As SldWorks.RefPlaneFeatureData  
Dim Feature As SldWorks.Feature  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
  
    boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)  
    Set Feature = swSelMgr.GetSelectedObject5(1)  
    Set swRefPlane = Feature.GetDefinition  
    swRefPlane.AccessSelections swPart, Nothing  
    Debug.Print "Original offset distance: " & swRefPlane.Distance  
    swRefPlane.Distance = 0.1  
    Debug.Print "Modified offset distance: " & swRefPlane.Distance  
    Feature.ModifyDefinition swRefPlane, swPart, Nothing  
  
End Sub
```

Modify Surface-cut Feature Example (C#)

This example shows how to modify a surface-cut feature.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Flips the direction of the surface-cut feature.
// 2. Examine the Immediate window.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SurfCutFeatureCSharp.csproj
{
    partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            SurfCutFeatureData swSurfCutFeature = default(SurfCutFeatureData);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\SurfCut.sldprt";
            swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Get the surface-cut feature
            status = swModelDocExt.SelectByID2("SurfaceCut1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSurfCutFeature = (SurfCutFeatureData)swFeature.GetDefinition();
            status = swSurfCutFeature.AccessSelections(swModel, null);

            // Flip direction of surface cut
            swSurfCutFeature.Flip = true;
            Debug.Print("Surface-cut feature flipped: " + status);

            // Update definition of feature
            swFeature.ModifyDefinition(swSurfCutFeature, swModel, null);

            // Rebuild part
            swModel.EditRebuild3();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Modify Surface-cut Feature Example (VBA)

This example shows how to modify a surface-cut feature.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Flips the direction of the surface-cut feature.
' 2. Examine the the Immediate.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swSurfCutFeature As SldWorks.SurfCutFeatureData
Dim status As Boolean
Dim errors As Long, warnings As Long
Dim fileName As String

Sub main()

Set swApp = Application.SldWorks
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SurfCut.sldprt"
swApp.OpenDoc6 fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings
Set swModel = swApp.ActiveDoc
Set swModelDocExt = swModel.Extension

' Get the surface-cut feature
status = swModelDocExt.SelectByID2("SurfaceCut1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swSelMgr = swModel.SelectionManager
Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
Set swSurfCutFeature = swFeature.GetDefinition
status = swSurfCutFeature.AccessSelections(swModel, Nothing)

' Flip direction of surface cut
swSurfCutFeature.Flip = True
Debug.Print ("Surface-cut feature flipped: " & status)

' Update definition of feature
swFeature.ModifyDefinition swSurfCutFeature, swModel, Nothing

' Rebuild part
swModel.EditRebuild3

End Sub
```

Modify Surface-cut Feature Example (VB.NET)

This example shows how to modify a surface-cut feature.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Flips the direction of the surface-cut feature.
' 2. Examine the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swSurfCutFeature As SurfCutFeatureData
        Dim status As Boolean
        Dim errors As Integer, warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SurfCut.sldprt"
        swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Get the surface-cut feature
        status = swModelDocExt.SelectByID("SurfaceCut1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swSurfCutFeature = swFeature.GetDefinition
        status = swSurfCutFeature.AccessSelections(swModel, Nothing)

        ' Flip direction of surface cut
        swSurfCutFeature.Flip = True
        Debug.Print("Surface-cut feature flipped: " & status)

        ' Update definition of feature
        swFeature.ModifyDefinition(swSurfCutFeature, swModel, Nothing)

        ' Rebuild part
        swModel.EditRebuild3()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Move and Copy Body by Setting Transforms Example (VBA)

This example shows how to move and copy bodies by setting transforms.

```
'-----  
'  
' Preconditions:  
' (1) Part document is open.  
' (2) Body-Move/Copy1 feature exists.  
'  
' Postconditions: Body is moved and copied as per transform settings.  
'  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim part As SldWorks.PartDoc  
    Dim component As SldWorks.Component2  
    Dim moveCopyFeat As SldWorks.feature  
    Dim moveCopy_featData As SldWorks.MoveCopyBodyFeatureData  
    Dim boolstatus As Boolean  
  
    Set swApp = Application.SldWorks  
    Set part = swApp.ActiveDoc  
  
    Set moveCopyFeat = part.FeatureByName("Body-Move/Copy1")  
    Set moveCopy_featData = moveCopyFeat.GetDefinition  
    boolstatus = moveCopy_featData.AccessSelections(part, component)  
  
    moveCopy_featData.TransformType = swTransformType_Translation  
    moveCopy_featData.TransformX = 0.02  
    moveCopy_featData.TransformY = 0.03  
    moveCopy_featData.TransformZ = 0.04  
  
    boolstatus = moveCopyFeat.ModifyDefinition(moveCopy_featData, part, Nothing)  
    moveCopy_featData.ReleaseSelectionAccess  
  
End Sub
```

Modify Move/Copy Body Using Vertex Example (C#)

This example shows how to modify a move/copy body feature using a vertex.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects a solid body and two vertices.
// 3. Inserts a move/copy body feature and gets and sets some of its feature data.
// 4. Examine the FeatureManager design tree, the graphics area, and
//    the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace MoveCopyBodyFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            MoveCopyBodyFeatureData swMoveCopyBodyFeatureData = default(MoveCopyBodyFeatureData);
            Vertex swVertexFrom = default(Vertex);
            Vertex swVertexTo = default(Vertex);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;

            //Select solid body and vertices for move/copy body feature
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, true, 4, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, true, 8, null, 0);

            //Insert move/copy body feature
            swFeature = (Feature)swFeatureManager.InsertMoveCopyBody2(0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, true, 1);

            //Roll forward the feature and get and set move/copy body feature data
            swMoveCopyBodyFeatureData = (MoveCopyBodyFeatureData)swFeature.GetDefinition();
            status = swMoveCopyBodyFeatureData.AccessSelections(swModel, null);
            Debug.Print("Move/copy body feature data:");
            Debug.Print(" Number of bodies to move or rotate and copy: " + swMoveCopyBodyFeatureData.GetBodiesCount());
            Debug.Print(" Number of copies: " + swMoveCopyBodyFeatureData.NumberOfCopies);
            Debug.Print(" Transform type (0 = None, 1 = Translation, 2 = Rotation): " + swMoveCopyBodyFeatureData.TransformType);

            //Change translation vertex
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, true, 8, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swVertexFrom = (Vertex)swSelectionMgr.GetSelectedObject6(1, -1);
            swMoveCopyBodyFeatureData.TransformReferenceEntity = swVertexFrom;
            swMoveCopyBodyFeatureData.TranslateToVertex = swVertexFrom;
            status = swModelDocExt.SelectByID2("", "VERTEX", 0, -0.065, 0, false, 0, null, 0);
            swVertexTo = (Vertex)swSelectionMgr.GetSelectedObject6(1, -1);
            swMoveCopyBodyFeatureData.TranslateToVertex = swVertexTo;
            swMoveCopyBodyFeatureData.TransformX = 0.05;

            //Apply the changes and roll back the feature
            swFeature.ModifyDefinition(swMoveCopyBodyFeatureData, swModel, null);

            swModel.ViewZoomToFit2();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```